

BTech Major Project EndSem Evaluation
Department of Information Technology



NATIONAL INSTITUTE OF TECHNOLOGY KARNATAKA SURATHKAL

Learning-Based Log Anomaly Detection, Cryptanalysis, and Design of Image Sharing and Encryption Schemes

Nithin S | 221IT085

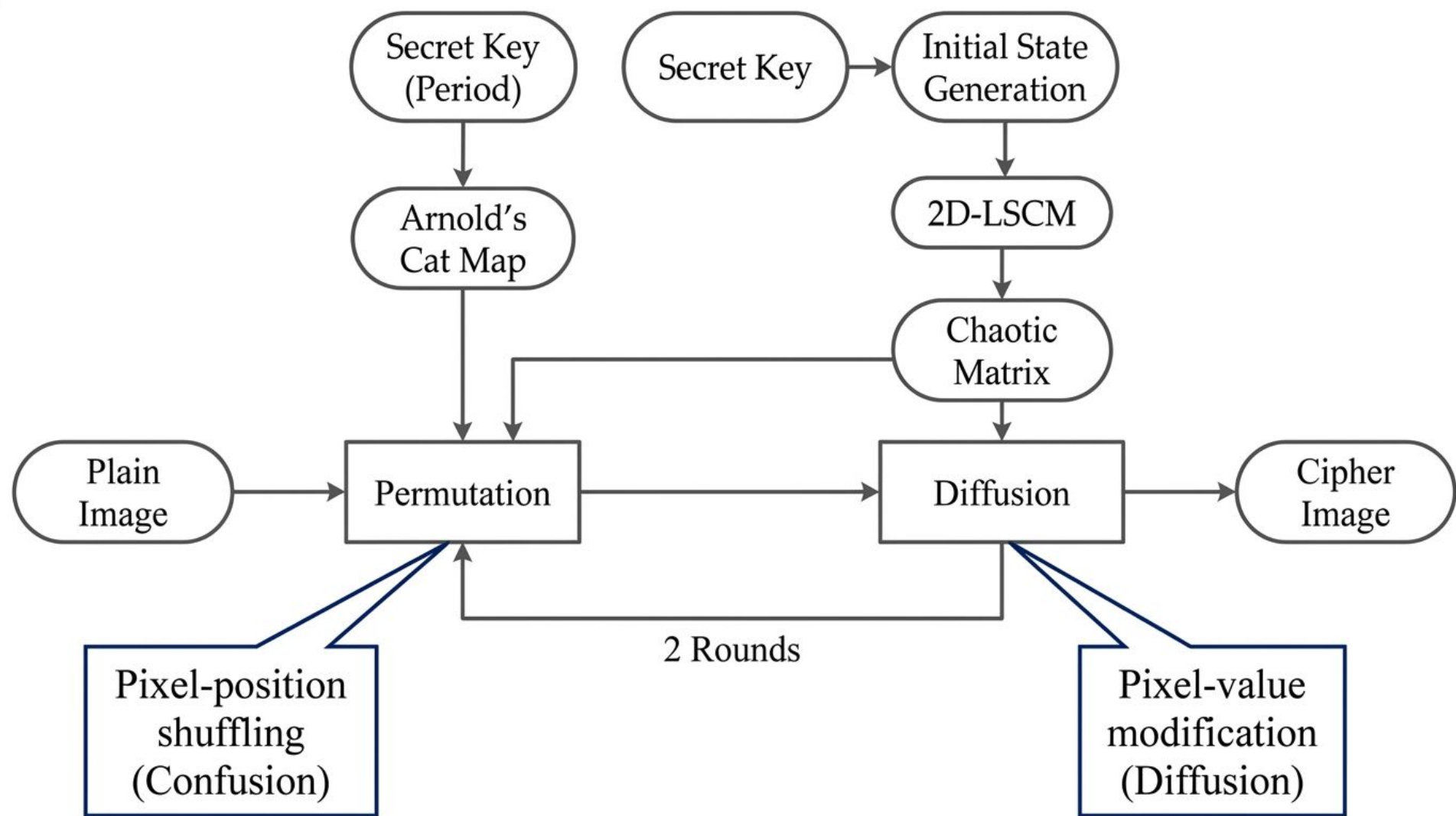
Sanketh NS | 221IT060

Vrushank TS | 221IT083

Cryptanalysis of a Medical Image Encryption Scheme

Dissecting a Chosen-Plaintext Attack
on a Multiple Chaotic Map Architecture





The Permutation Engine

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 2 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} \pmod{n}$$



Reorders pixel
coordinates
deterministically.
Periodicity: $3n+1$.

The Diffusion Engine

$$x_{i+1} = \sin \left(\pi \left(\begin{matrix} 4\theta x_i(1 - x_i) + \\ (1 - \theta) \sin(\pi y_i) \end{matrix} \right) \right)$$



Generates the
pseudorandom
keystream X_i .
206-bit key space.

The Core Vulnerability: Three Fatal Flaws

1. Plaintext-Independent Keystream

The keystream X_i is generated solely from the secret key.

Functionally equivalent to a reusable one-time pad.

2. Decoupled Stages

Permutation operates completely independently of diffusion.

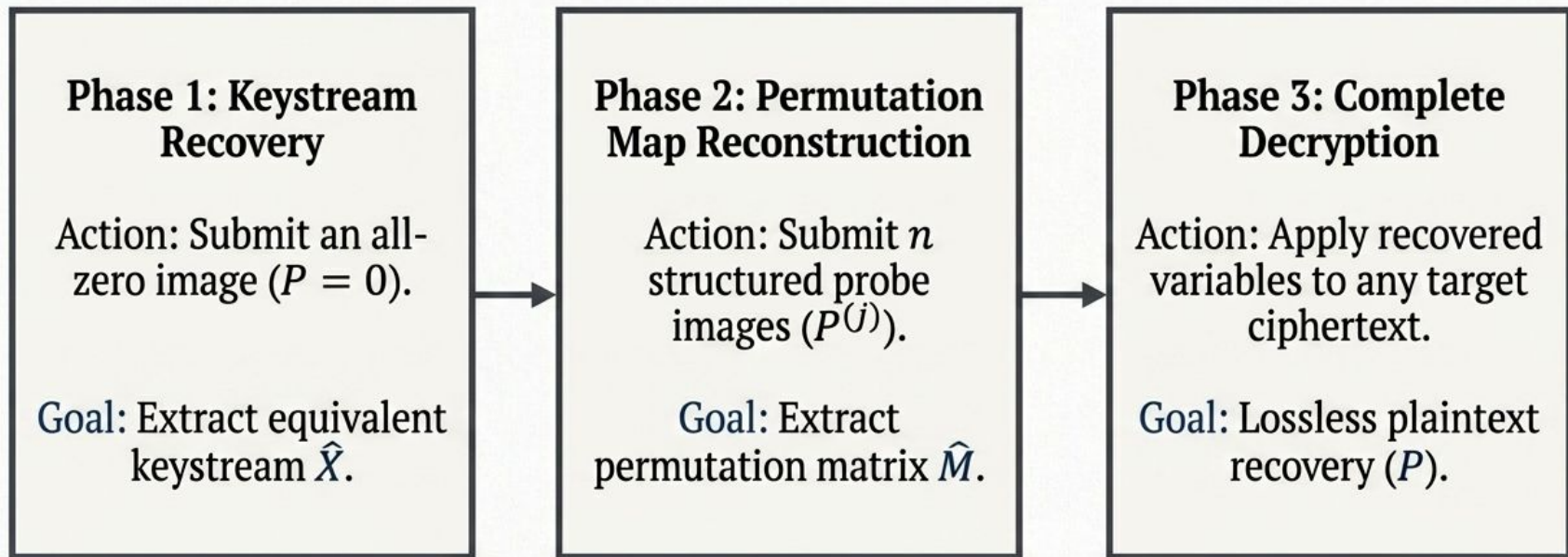
Allows an attacker to isolate and invert each layer separately.

3. Publicly Invertible Preprocessing

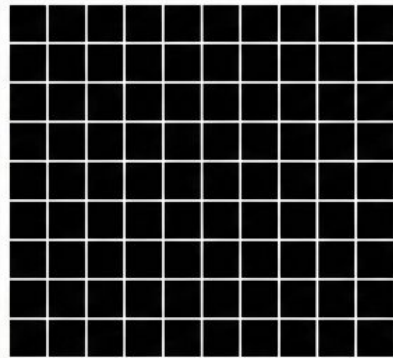
Arnold's Cat Map uses no secret parameters except the iteration count P_i .

Can be reversed mathematically by the attacker before reaching the oracle.

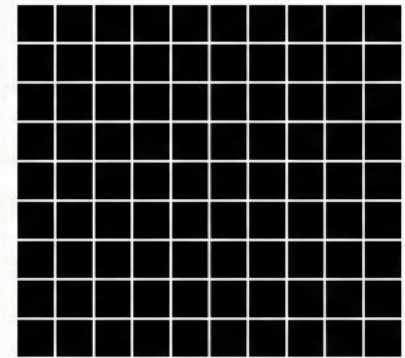
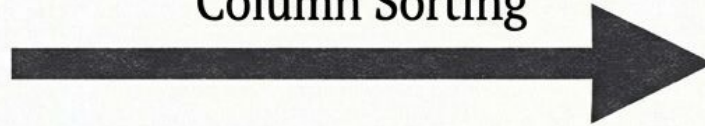
The Attack Strategy Overview: Chosen-Plaintext Attack (CPA)



Phase 1: Keystream Recovery Setup



Arnold's Cat Map
Column Sorting



$ACM_{P_i}(0) = 0$
(Arnold's Cat Map of 0 is 0)

$T^{(0)} = 0$
(Permuted Image of 0 is 0)

Because every pixel value is zero, shuffling their coordinates has zero effect. The entire permutation layer is bypassed, sending a pristine zero-matrix directly into the diffusion layer.

Phase 1 Math: The Diffusion Collapse

$$C_i^{(0)} = \cancel{T_i^{(0)}}^0 + C_{i-1}^{(0)} + C_{i-2}^{(0)} + X_i \pmod{F}$$



$$X_i = (C_i^{(0)} - C_{i-1}^{(0)} - C_{i-2}^{(0)}) \pmod{F}$$

With the permuted plaintext zeroed out, the resulting ciphertext consists strictly of the previous cipher block and the raw keystream. A single oracle query yields all 65,536 values of the keystream $\hat{X}$.

Phase 2: Permutation Map Reconstruction

	Col 1	Col 2	...	Column j	...	Col m
Row 1	0	0	...	0	...	0
Row 2	0	0	...	1	...	0
...	0	0	...	2	...	0
Row r	0	0	...	...	...	0
...	0	0	...	...	...	0
Row n	0	0	0	$n-1$	...	0

$$P_{r,c}^{(j)} = \begin{cases} r, & c = j \\ 0, & c \neq j \end{cases}$$

To isolate the column-wise permutation, the attacker crafts a set of probe images. Each probe encodes an ordered index sequence in a single column, with all other columns blanked out.

Phase 2 Math: Pre-Canceling the Arnold Map

Step 1: The Pre-image

$$Q^{(j)} = ACM^{-P_i}(P^{(j)})$$

The attacker pre-applies the inverse Arnold map to the probe.



Step 2: The Oracle

$$S^{(j)} = ACM_{P_i}(Q^{(j)})$$

When submitted, the encryption oracle applies its own forward Arnold map.



Step 3: The Cancellation

$$S^{(j)} = ACM_{P_i}(ACM^{-P_i}(P^{(j)})) = P^{(j)}$$

The two operations perfectly cancel each other out. The oracle unwittingly restores the original structured probe, feeding it directly into the 2D-LSCM sorting mechanism.

Phase 2 Math: Reading the Permutation

$$\hat{T}^{(j)} = D_{\hat{X}}(C^{(j)})$$

Inverse Diffusion operation

$$\hat{\sigma}_j(\ell) = \hat{T}^{(j)}[\ell, j]$$

Permutation Extraction

Because we recovered $\hat{X}$ in Phase 1, we can run the diffusion backward on the oracle's output. The resulting matrix column contains the exact indices of the hidden chaotic permutation $\hat{\sigma}_j$.

Cost: n oracle queries (256 queries for a 256×256 image) to reconstruct the entire permutation map $\hat{M}$.

Phase 3: Complete Decryption Algorithm

Total System Collapse: Decrypting Target Ciphertexts

1. Inverse Diffusion

- Apply recovered keystream $\hat{X}$ to ciphertext C . Result: $\hat{T}$.

2. Inverse Column Permutation

- Reverse the 2D-LSCM sort using recovered map $\hat{M}$. Result: $\hat{S}$.

3. Inverse Arnold's Cat Map

- Apply Arnold's map $3n + 1 - P_i$ times. Result: Exact Plaintext $\hat{P}$.

The secret keys (K_1, K_2) are entirely bypassed.

Claimed Security

Actual Vulnerability

206-bit (>100 \$-bit minimum standard).

Bypassed entirely.
Zero bits of effective security.

Resistant to
Differential/CPA attacks.

Broken in exactly $n + 1$ queries
(257 queries for a standard
medical image).

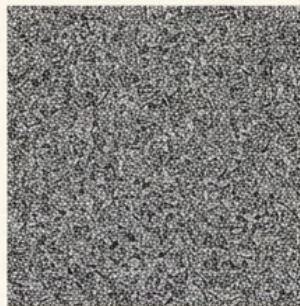
Lossless recovery via
secret key only.

100% exact CPA recovery.
Max Absolute Deviation = 0.

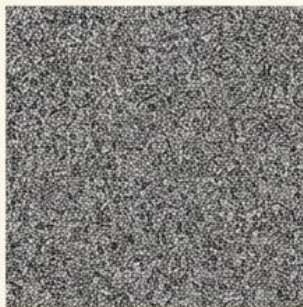
Visual Proof: The Decryption in Action



(a) Original Target Plaintext (P)



(b) Encrypted Output (Sent to Oracle)



(c) Blank Cipher $C^{(0)}$ (Yields Keystream)



(d) CPA-Decrypted $\hat{P}$ (Zero Error, No Key)

This paper was submitted to the journal of Cryptologia

A Cryptanalysis of Multi-Chaos-Based Image Encryption for IoT Using Chosen Plaintext Attacks

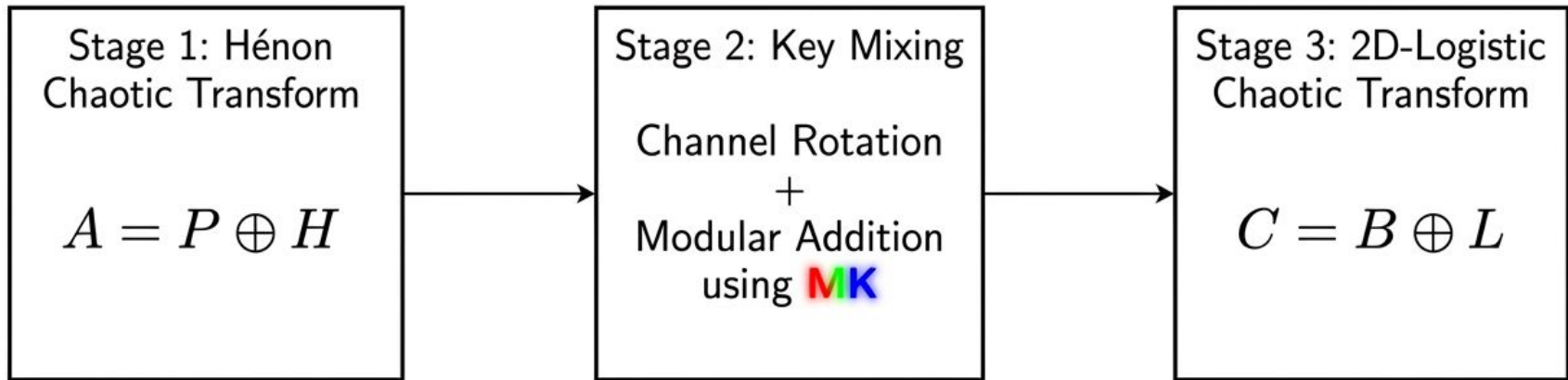
A Mathematical Proof of Chosen-Plaintext Vulnerability
in the MMCBE Cryptosystem.

$O(N)$ Complexity | 256 Oracle Queries | 100% Plaintext Recovery

Statistical
Illusion

- ✓ NPCR $\approx 99\%$
- ✓ UACI $\approx 33\%$
- ✓ Shannon Entropy ≈ 8

**Cryptographic failure
occurs when the
ciphertext distribution
is decoupled from the
plaintext input.**



The process appears chaotic, but structurally, it is a linear sequence of XORs and modular additions.

Target Ciphertext
(Red Channel)

Input Plaintext
(Green Channel)

$$C_R[k] = [(P_G[k] \oplus H[k]) + MK_G[\ell]] \bmod 256 \oplus L[k]$$

Hénon Sequence

Key Mixing Array

Logistic Sequence

By isolating a single color channel at a specific pixel position k , the full multi-map encryption reduces to elementary algebra.

$$C_R[k] = [(P_G[k] \oplus H[k]) + MK_G[\ell]] \bmod 256 \oplus L[k]$$

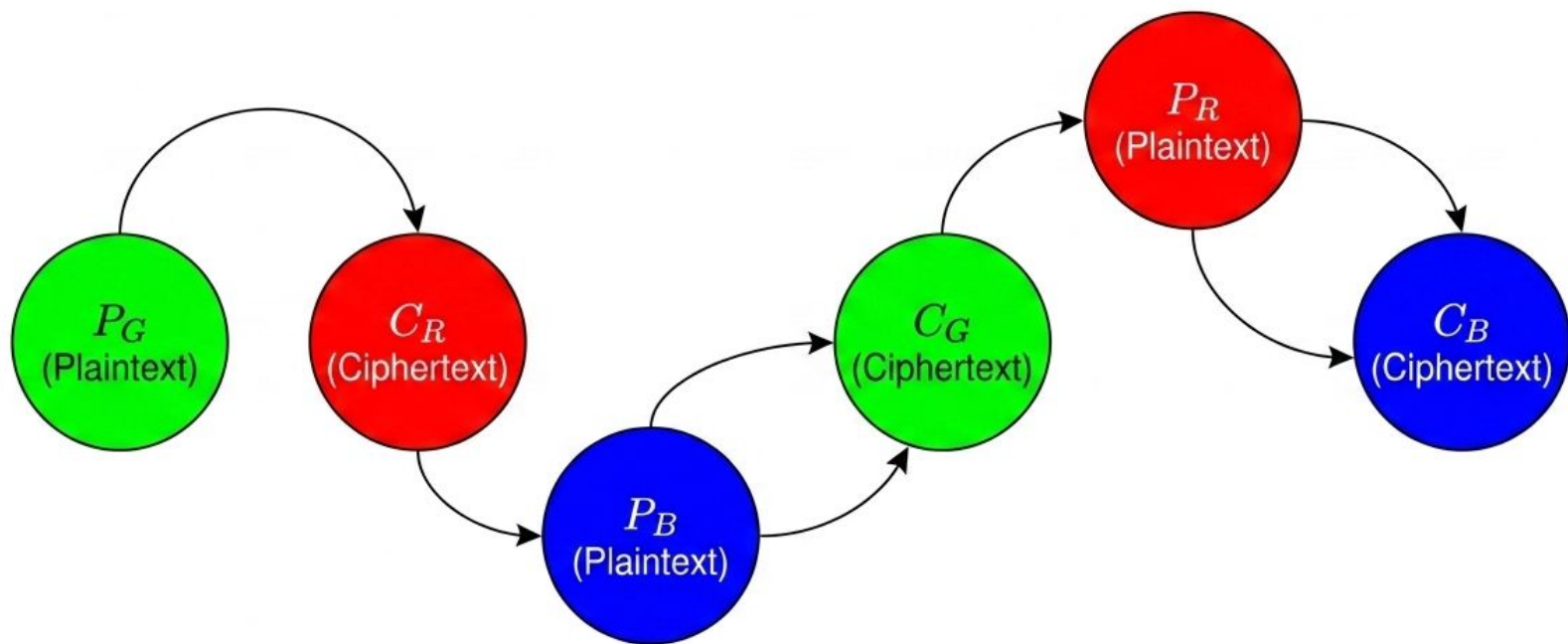


CONSTANTS



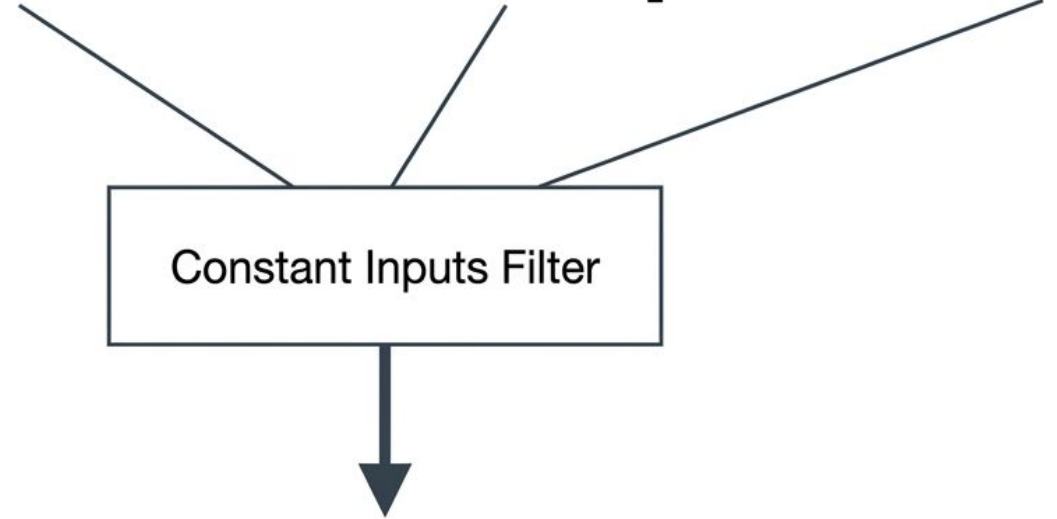
Initial Seeds:
 (x_H, y_H) and
 (x_L, y_L, R)

Defect 1: Plaintext Independence. Both chaotic keystreams are generated purely from the fixed 512-bit master key. Because they never interact with the plaintext, $H[k]$ and $L[k]$ are entirely deterministic. If the key is static, the “chaos” is a constant.



Defect 2: The Cross-Coupling Loop. The key mixing stage rigidly rotates the color channels. This deterministic rotation guarantees that the Red ciphertext component exclusively encodes the Green plaintext component.

$$C_R[k] = [(P_G[k] \oplus H[k]) + MK_G[\ell]] \bmod 256 \oplus L[k]$$

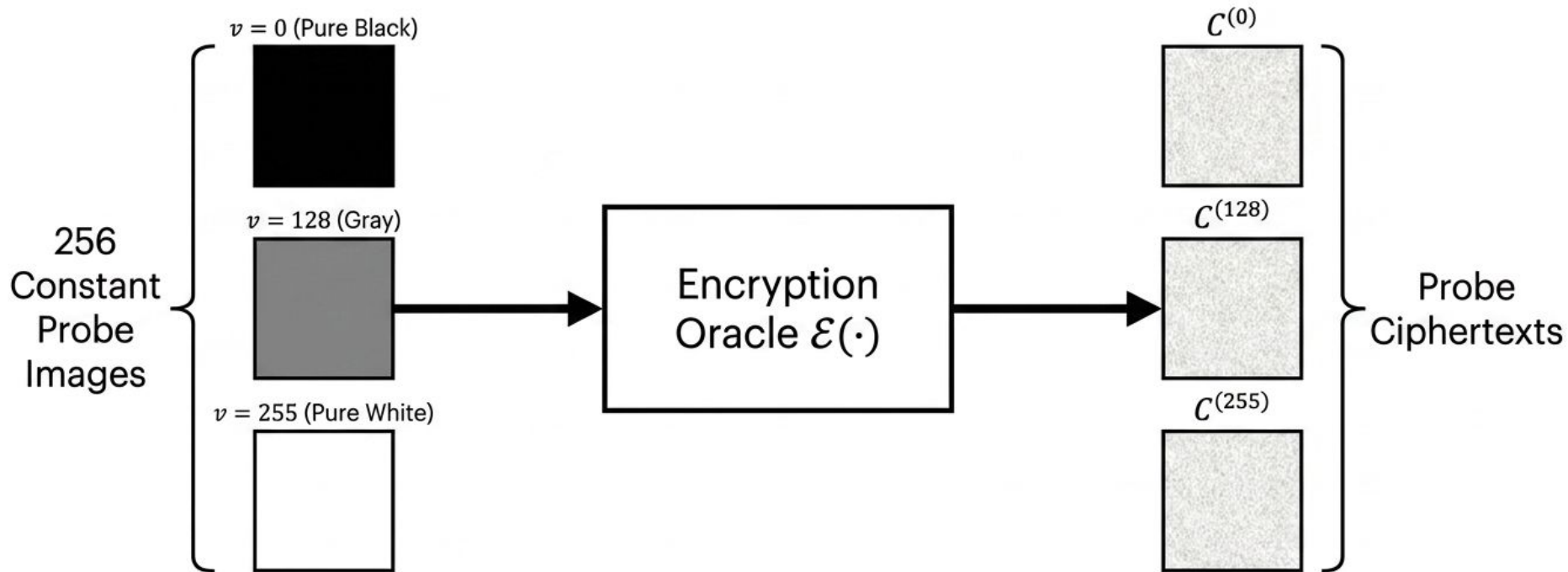


Constant Inputs Filter

$$C_R[k] = f(v) \text{ where } v \text{ is a constant}$$

The Synthesis. Because the equation is deterministic (Defect 1) and bijective (no inter-pixel diffusion), complex encryption collapses into simple algebra. The system functions as a static, $256 \times N$ lookup table. An attacker does not need to crack the key; they only need to map the table.

Attack Phase 1: Table Construction



Attack Phase 1: Table Construction. By sending 256 spatially uniform, constant-value images into the oracle, we capture the exact ciphertext response for every possible pixel value. No key required.

Pixel-wise Channel Matching

Recovered Plaintext $\hat{P}$

$$\hat{P}_{\textcolor{green}{G}}[k] = v \quad \text{such that} \quad C_{\textcolor{red}{R}}^{(v)}[k] = C_{\textcolor{red}{R}}^*[k]$$

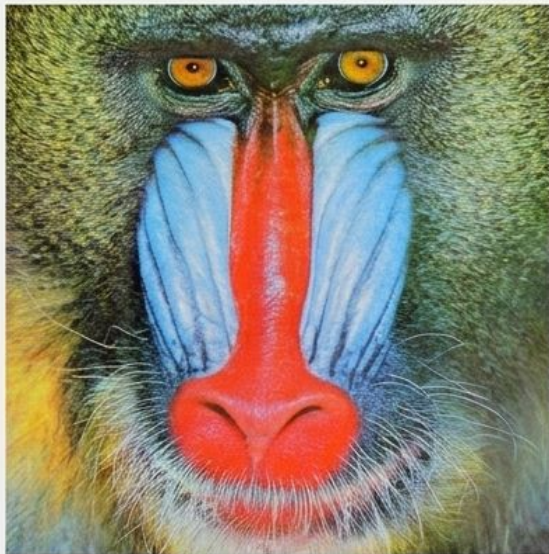
$$\hat{P}_{\textcolor{blue}{B}}[k] = v \quad \text{such that} \quad C_{\textcolor{green}{G}}^{(v)}[k] = C_{\textcolor{green}{G}}^*[k]$$

$$\hat{P}_{\textcolor{red}{R}}[k] = v \quad \text{such that} \quad C_{\textcolor{blue}{B}}^{(v)}[k] = C_{\textcolor{blue}{B}}^*[k]$$

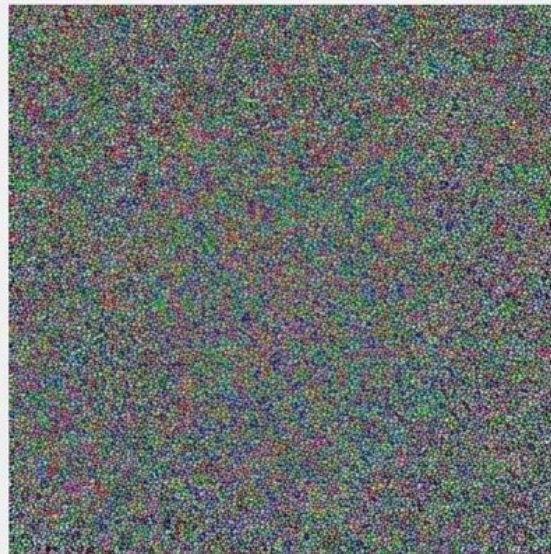
Attack Phase 2: Channel Matching. Utilizing the RGB cross-coupling loop, we compare the target ciphertext C^* against our 256 probe ciphertexts $C^{(v)}$. A direct equivalence match instantly yields the precise plaintext pixel.

[Blank]	Normal Encryption	CPA Exploit
Oracle Queries	—	256
Arithmetic Time Complexity	$O(N)$	$O(N)$
Key Knowledge Required	N/A	None
Match Accuracy	—	100%

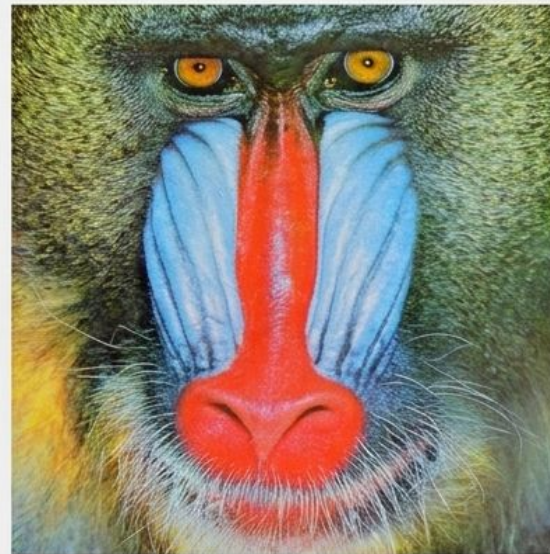
The Cost Matrix. The attack operates in $O(N)$ time, making the total exploit roughly equivalent in computational cost to encrypting a single image. Zero brute force is required.



Original Plaintext P



MMCBIE Ciphertext C^*



CPA Decrypted $\hat{P}$ ($\Delta = 0$)

Empirical Proof. Experimental simulation perfectly replicates the theory.
256 oracle queries result in a mathematically exact recovery of the plaintext.
Maximum pixel difference: 0.

Vulnerability 1: Plaintext Independence

Fix: Hash-Driven IV
Perturbation.

Mechanism: XOR the
plaintext hash $H(P)$ with
the chaotic initial states
to ensure unique
keystreams per image.

Vulnerability 2: Bijective Isolation

Fix: Chained Diffusion.

Mechanism:

$$C[k] = (B[k] \oplus L[k] \oplus C[k-1]) \bmod 256.$$

Ensure $C[k]$ depends on
preceding pixels to trigger
an avalanche effect.

Vulnerability 3: Linear Key Mixing

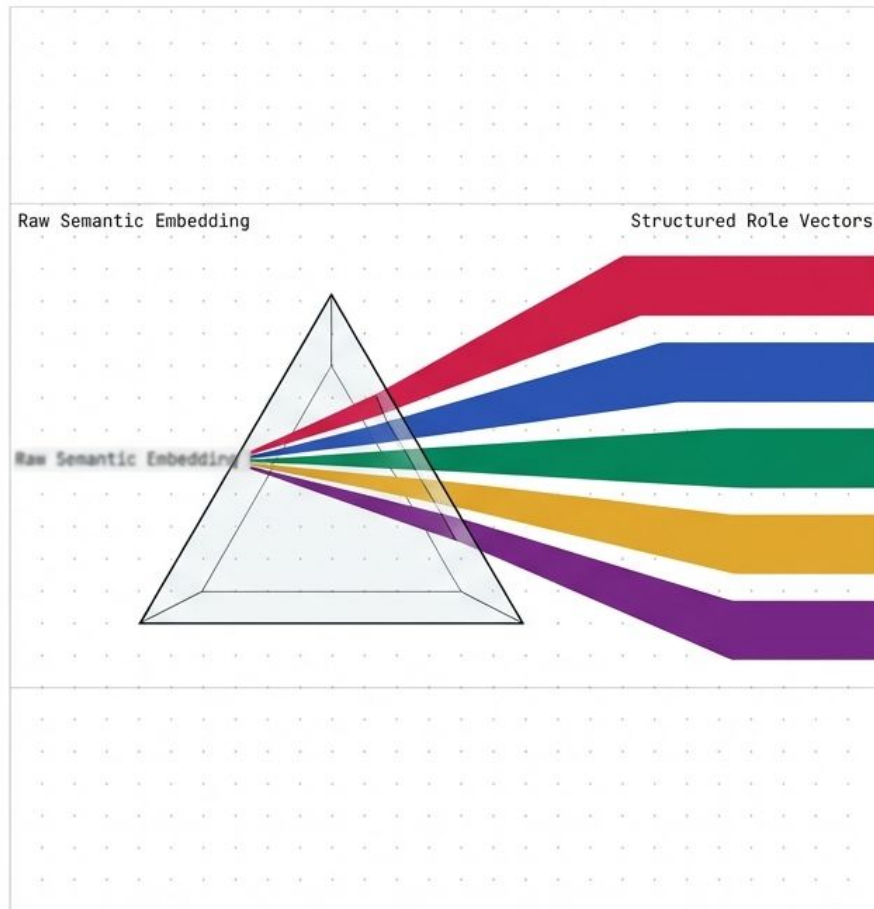
Fix: Non-Linear
Substitution.

Mechanism: Replace
predictable modular
addition with a Chaotic
Latin Square S-Box to
eliminate linear bijection.

This paper was submitted to the journal of Cryptologia

Role-Aware Log Representation

Improving Anomaly Detection
Through Latent Token Role
Induction



The image shows a terminal window with a black background and white text. Overlaid on the terminal output are large, bold, red characters. The top part of the overlay reads '10t' and the bottom part reads 'TB/day'. The background text is a log of database transactions, with entries like 'Connection (laid out) while reading', 'Transaction failed, 10: tx-7890xyz', and 'Transaction failed, 10: tx-7890xyz, reason: connection timed out'. The log entries are partially obscured by the red overlay.

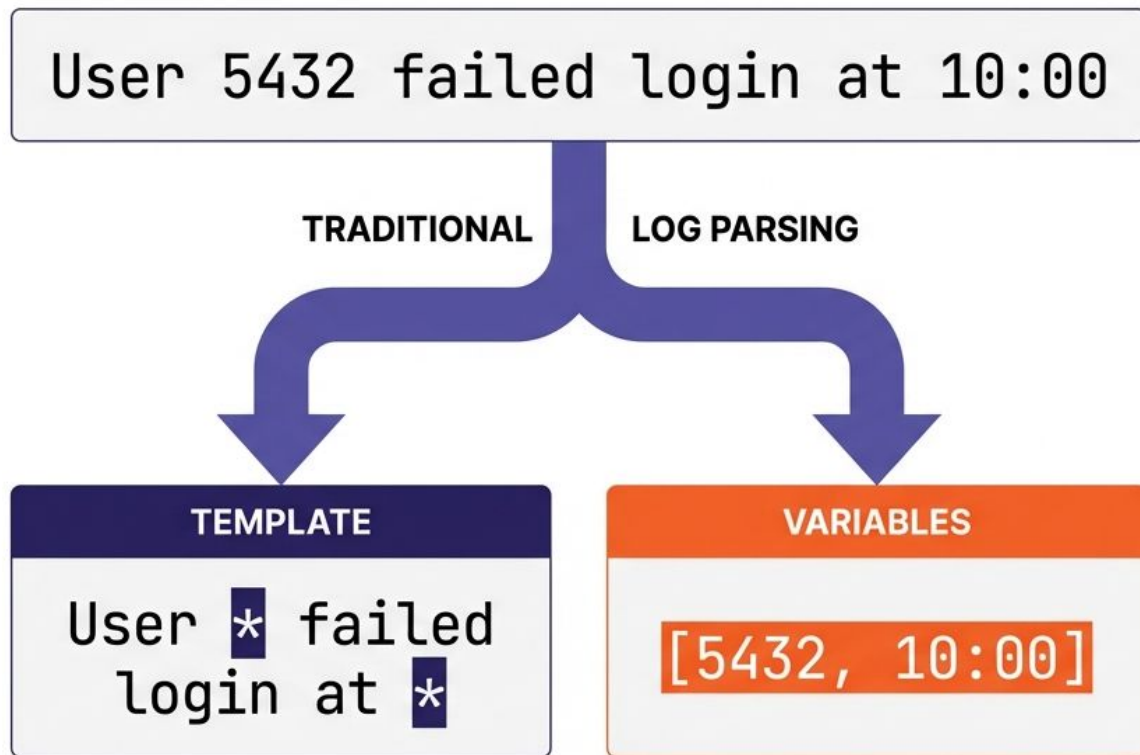
Why Manual Inspection is Impossible

Modern microservices and distributed cloud systems generate terabytes of logs daily. Systems produce millions of messages per second.

Semi-structured data. While templates are fixed, parameters like Block IDs, IPs, and locations change constantly.

Real-time detection is critical. A delay of minutes can lead to security breaches or cascading system failures.

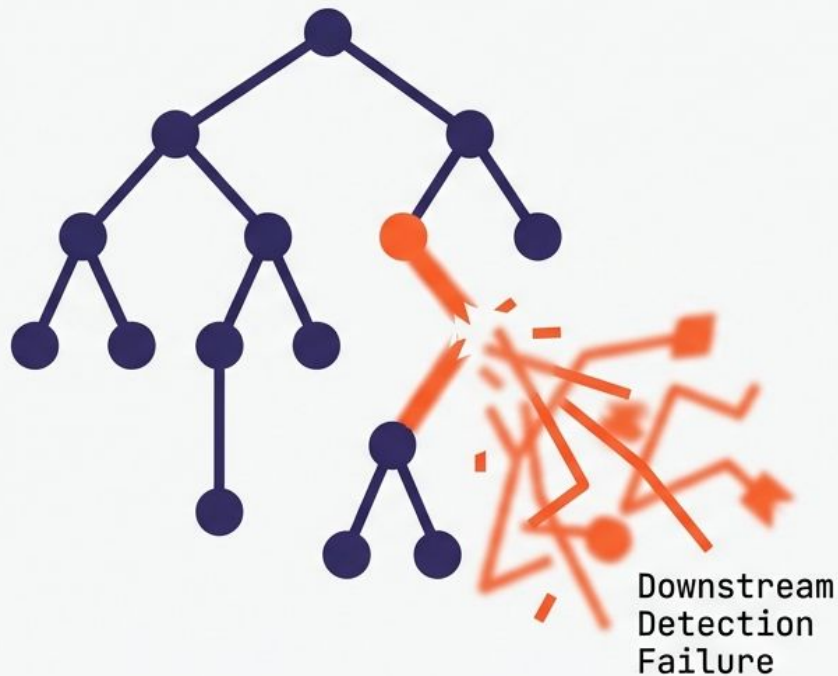
The Bottleneck of Log Analysis



Historical Approach: Frequent Pattern Mining

- Algorithms: SLCT, LogCluster, IPLoM
- Assumption: Constant tokens appear more frequently than variables.
- Failure Mode: Sensitive to parameter tuning; breaks on diverse logs.

The High Cost of Parsing Errors



14% – 89%

Parser Error Rates on Complex Datasets (BGL, Thunderbird)

- Technique: Fixed-depth tree parsing (e.g., Drain).
- Consequence: If the parser fails, the detector is blind.
- Even neural parsers (Logram) struggle with OOV tokens.

The Evolution of Log Analysis: A Trade-off



The Semantic Leap



Era 1: Parsing-Based (DeepLog, LogRobust)

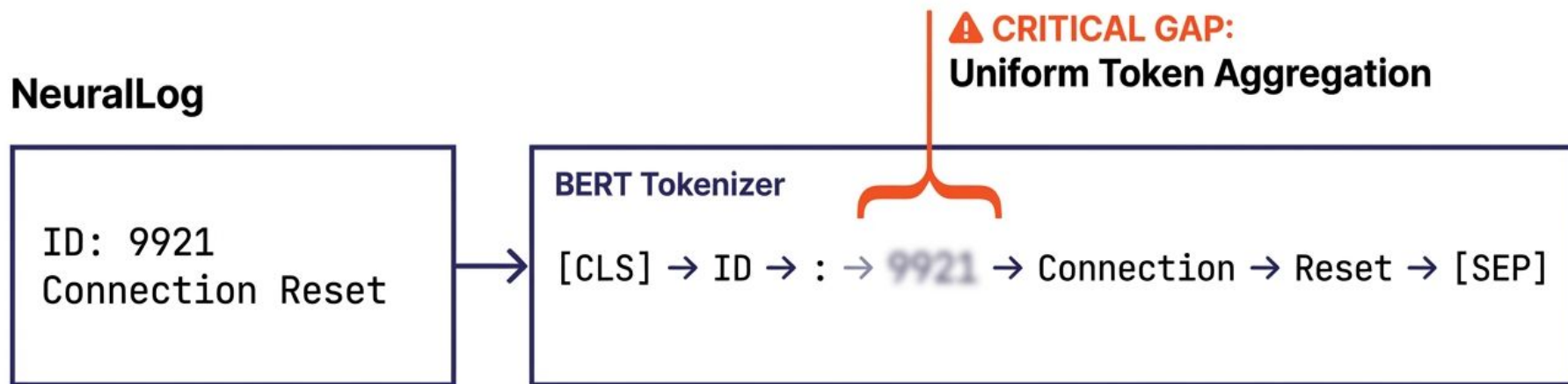
- **Method:** Extract Templates + Variables
- **Flaw:** High brittleness. Parser error rates 14-89%.
- **Consequence:** One parsing error = Detection failure.

Era 2: Parsing-Free (NeuralLog)

- **Method:** BERT encoding of raw text.
- **Win:** Robust to format changes. F1 > 0.95.
- **Flaw:** Structural Blindness (The Flattening).

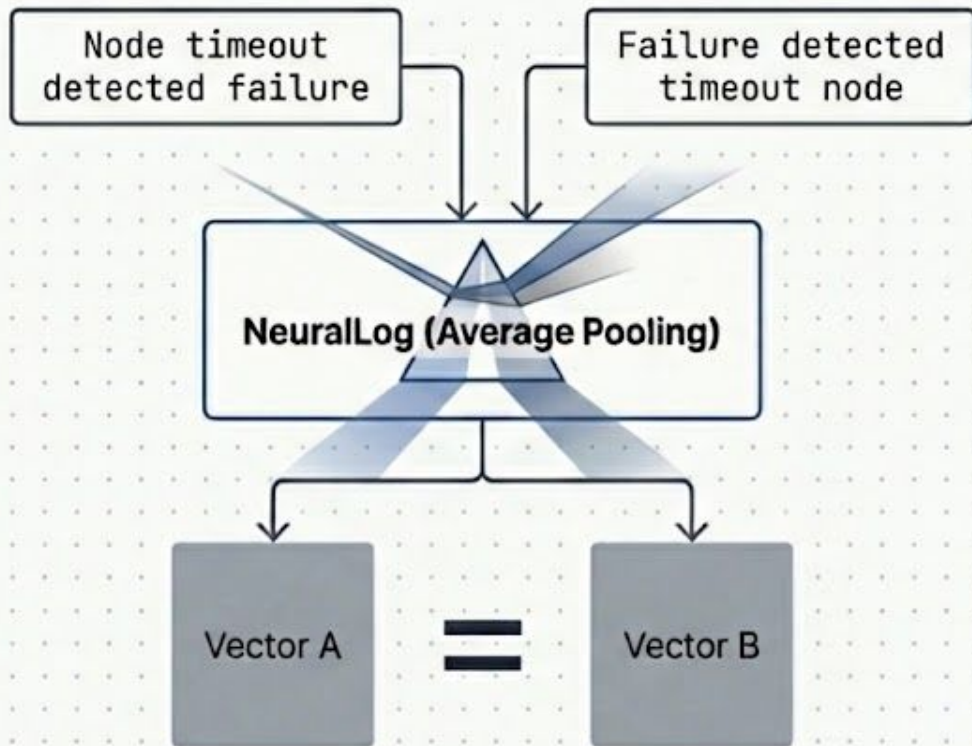
The Frontier: Parsing-Free Representation

NeuralLog



- Current State: NeuralLog encodes raw text, removing the parser.
- The Problem: Treats all tokens equally. Loss of numeric signals and structural roles.
- Need: Role-Aware frameworks that distinguish System IDs from Timestamps.

The Problem in NeuralLog Paper: The Flattening Effect



- 1. **Uniformity:** Averaging tokens disregards word order and structure.
- 2. **Number Blindness:** Removing numerics loses context (Error 500 vs 200).
- 3. **Black Box:** High accuracy, but zero interpretability of *why* an anomaly occurred.

The Evaluation Arena: 34 Million Log Messages

HDFS

Hadoop Distributed
File System

**11.17 Million
Messages**

Label Type: Block-level
Context: Distributed
Storage System

BGL

Blue Gene/L
Supercomputer

**4.74 Million
Messages**

Label Type: Line-level
Context: Numeric-heavy,
Supercomputing

Thunderbird

Sandia National Labs

**10 Million
Messages**

Label Type: Line-level
Context: High Ambiguity,
Uneven Types

Spirit

Supercomputer Logs

**7.98 Million
Messages**

Label Type: Line-level
Context: Large Scale
Infrastructure

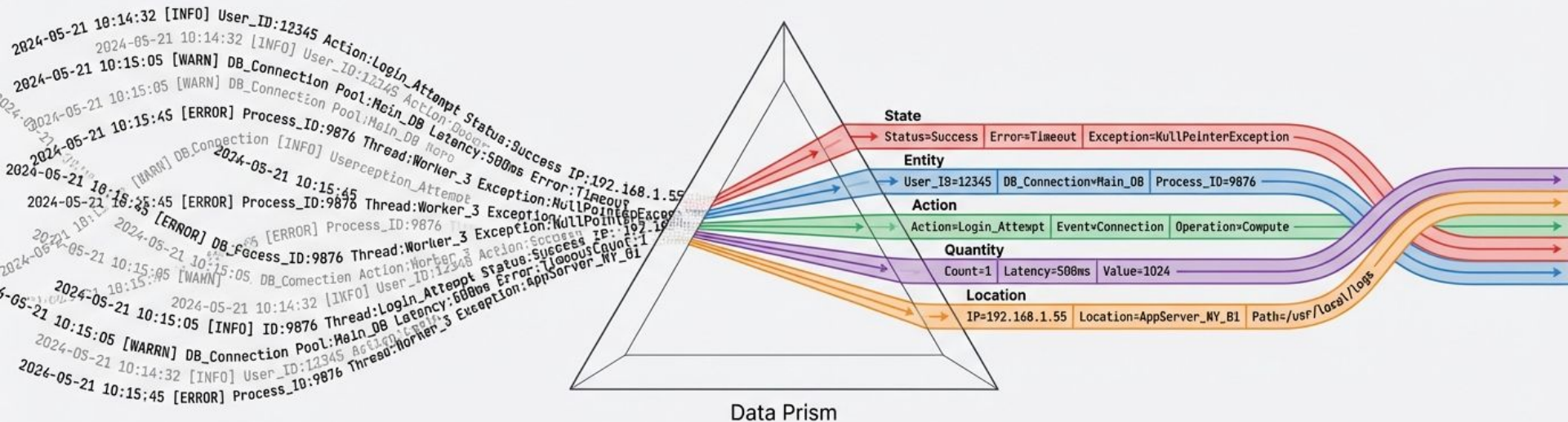
The Insight: Tokens Have Functional Roles



Hypothesis: Log tokens aren't just semantic text; they perform specific structural jobs. We must model these these 5 latent roles explicitly.

Turning Unstructured Log Noise into Semantic Structure

A Role-Aware Latent Token Induction Framework for Anomaly Detection



The Context

System health relies on logs, but raw logs are numerically dense and unstructured sequences.

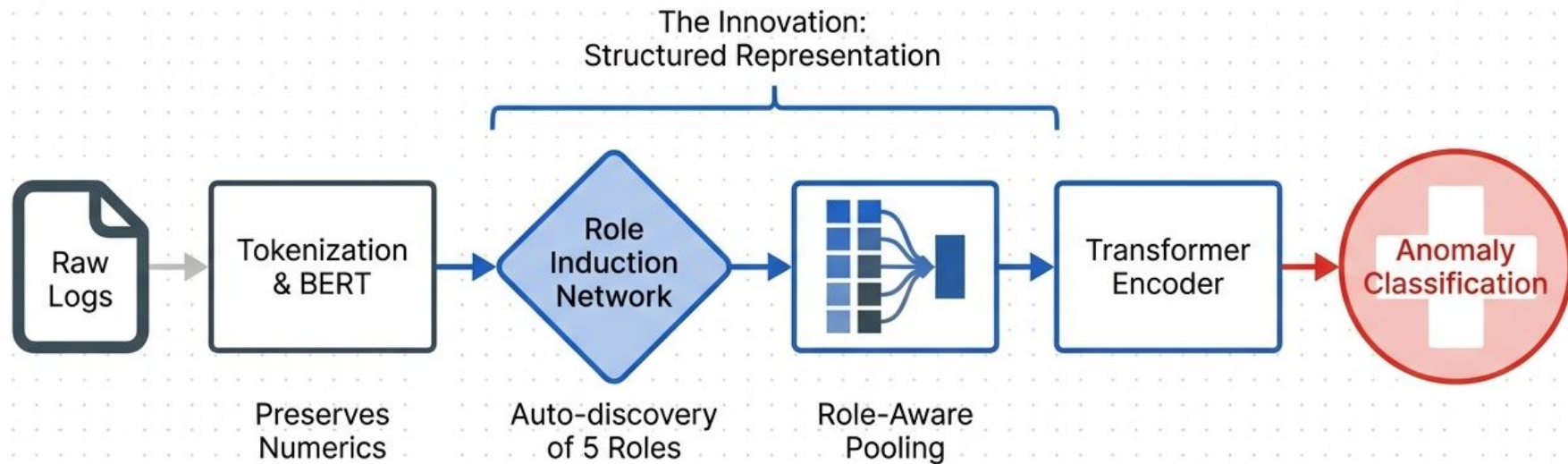
The Problem

Traditional anomaly detection averages these signals into a blur, losing critical numeric fidelity.

The Solution

We introduce a framework that identifies the functional role of every token—State, Entity, Action, Quantity, or Location—to detect anomalies with precision.

The Role-Aware Framework



Traditional Methods (e.g., NeuralLog)

Raw log
Task ID: 404 failed at block 500



Processed
Task ID: [N] failed at block [N]

Critical Context Erasure: '404' and '500' are treated as generic placeholders.

Role-Aware Approach

Task ID: 404 failed at block 500
Quantity Quantity



Task ID: [N] failed at block [N]

Context Retention: '404' is identified as a specific Error Code; '500' is a specific Block ID.

The Blind Spot: Previous methods remove numeric tokens to simplify processing.

The Cost: A resource count like “memory=0” or “retries=100” becomes structurally identical to “memory=16” or “retries=1”.

Our Approach: We retain numeric tokens because numerical values act as distinct vocabulary items necessary for correlation.

Enhanced Preprocessing Preserves Digital Fidelity

Raw Input

[2023-10-27]
Task ID: 404
failed at
datablockscanner



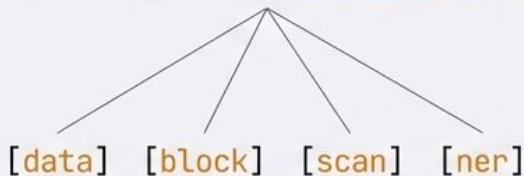
Normalization & Tokenization

task
id
:
404
failed
at
datablockscanner



WordPiece Decomposition (The Zoom-In)

datablockscanner



Handling Out-Of-Vocabulary (OOV) terms without losing meaning.

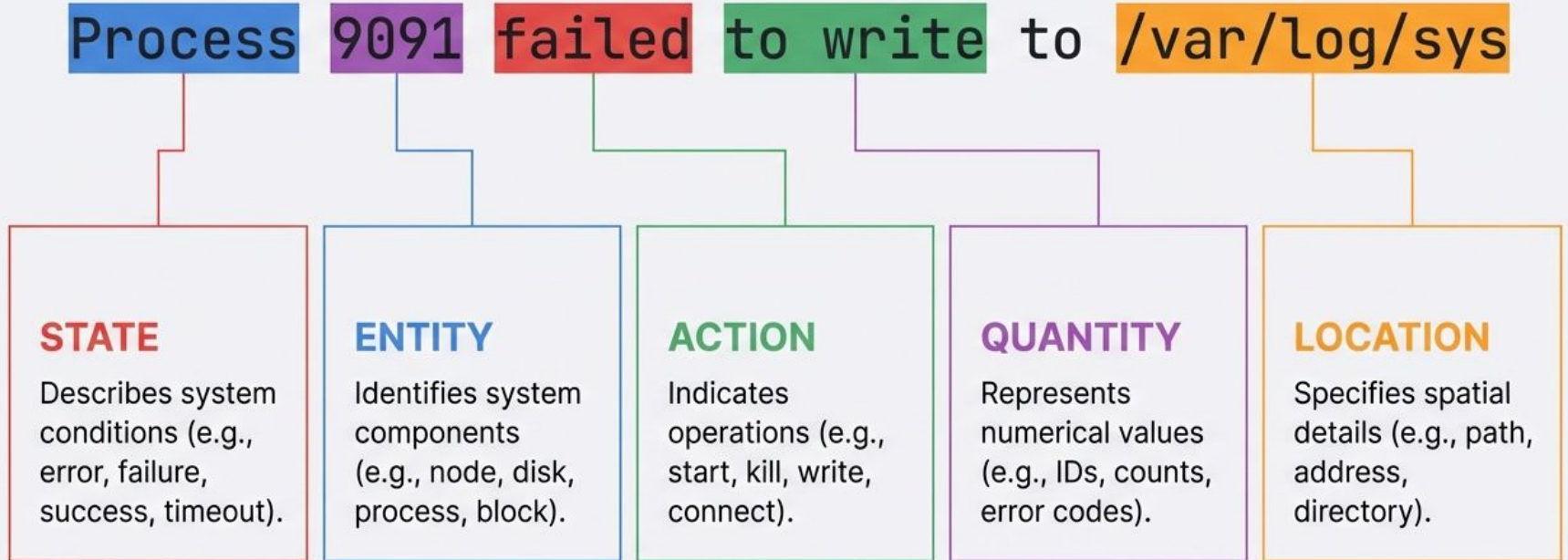
BERT-Base Embedding

Unlike standard methods that average subword embeddings, we utilize BERT-base (12 layers, 768 hidden units).

Contextual Integrity

Generates contextual embeddings that respect the integrity of rare technical terms.

The Five Latent Roles of Log Syntax

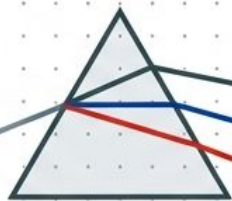


Mechanism I: Keeping the Numbers

Before & After

Standard Approach (NeuralLog)

HTTP [NUM] error
CPU usage [NUM]%
Block [NUM] failed



Our Approach

HTTP **404** error vs HTTP **500** error
CPU usage **100%**
Block **blk_1234** failed

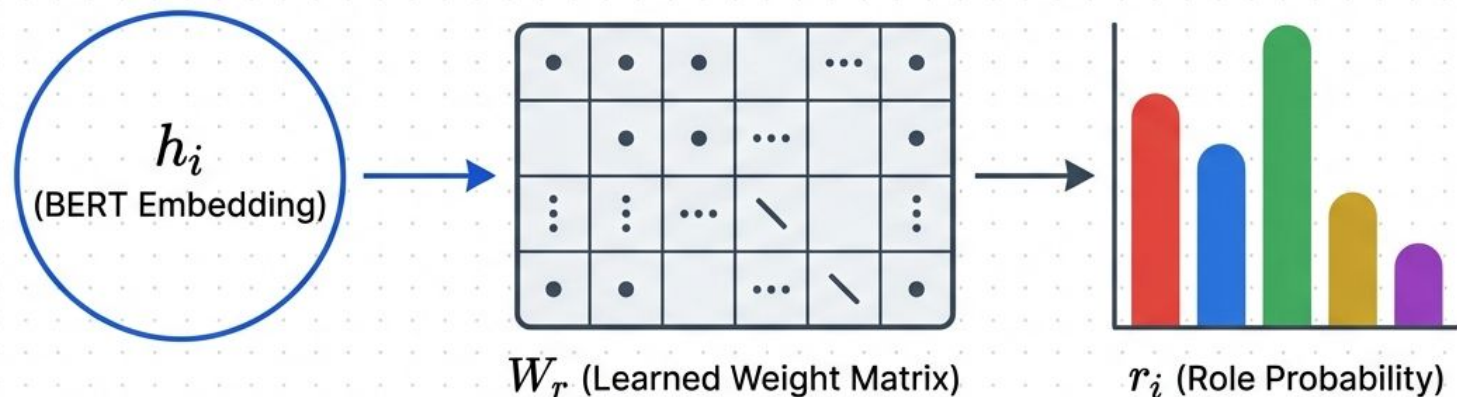
Loss of Critical Signal

Preservation of Context

Key Takeaways

- **Error Codes:** Distinguish failure modes.
- **Resource Utilization:** Distinguish idle (0%) from overload (100%).
- **Identifiers:** Enable correlation across events.

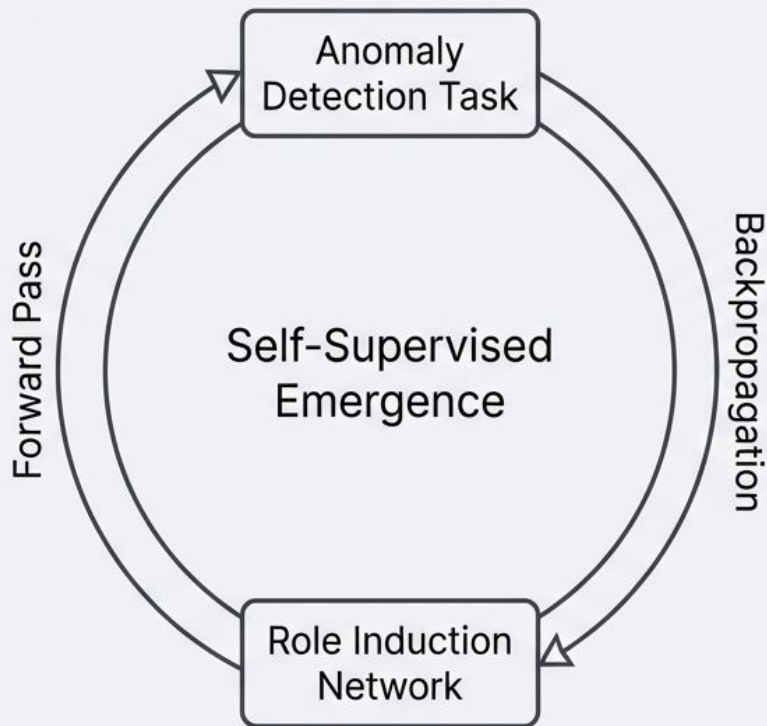
Mechanism II: Latent Token Role Induction



$$r_i = \text{softmax}(W_r \cdot \text{ReLU}(W_h h_i + b_h) + b_r)$$

- **Self-Supervised Learning:** The model learns to cluster tokens based on their contribution to the anomaly detection task.
- **No Manual Annotation:** The grammar is induced automatically from the data.

Learning Through Anomaly Detection, Not Annotation



- **Zero Manual Annotation:** We do not label tokens as 'State' or 'Action' by hand.
- **Functional Clustering:** Tokens that contribute similarly to detection (e.g., all tokens indicating failure states) naturally cluster together during training.
- **The Result:** The "roles" are learned behaviors, optimized specifically for finding system faults.

Fusing Semantics with Role Identity



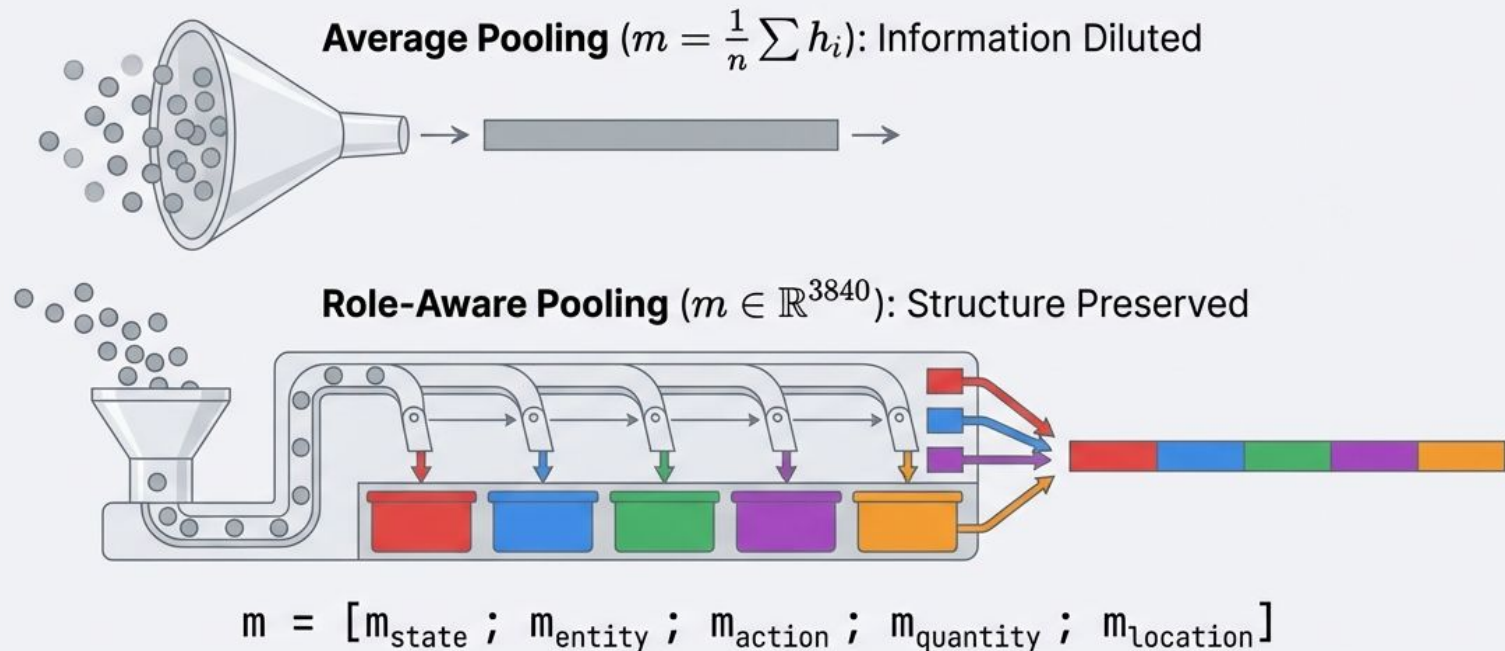
$$t_i = h_i \odot W_e r_i$$

We fuse the 'What' (Meaning) and the 'How' (Role) into a single representation.

W_e : Role Embedding Matrix ($\mathbb{R}^{768 \times 5}$)

The resulting token t_i carries both the context of the log message and the specific functional identity of the token.

Structured Pooling: Moving Beyond the Average



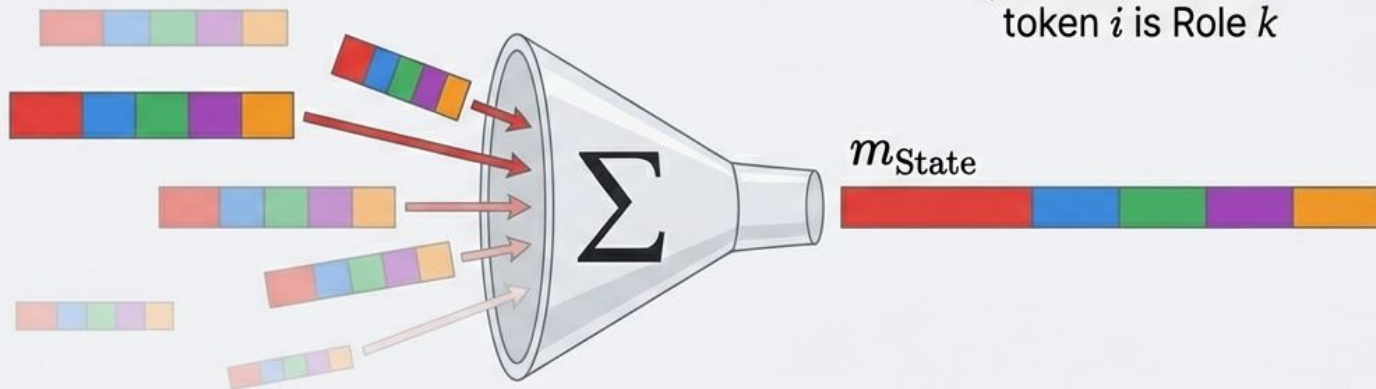
- NeuralLog simply averages embeddings, washing out signal.
- Our Method constructs a structured message vector where every dimension has a distinct purpose.
- Ensures a **Quantity** anomaly doesn't get hidden by a dominant **State** token.

The Mathematics of Role-Aware Aggregation

Segment for Role k
(e.g., **State**) $\rightarrow m_k = \frac{\sum_{i=1}^n r_{i,k} \cdot t_i}{\sum_{i=1}^n r_{i,k}}$

$\leftarrow$ Role-Aware Token

$\leftarrow$ Probability that token i is Role k



We compute a weighted average for each role. If a token has a low probability of being an **'Entity'**, it contributes almost nothing to the **Entity** segment.

Result: A precise, segmented representation of the log message.

Mechanism III: Role-Aware Pooling

Average Pooling



768 Dimensions (Blended Signal)



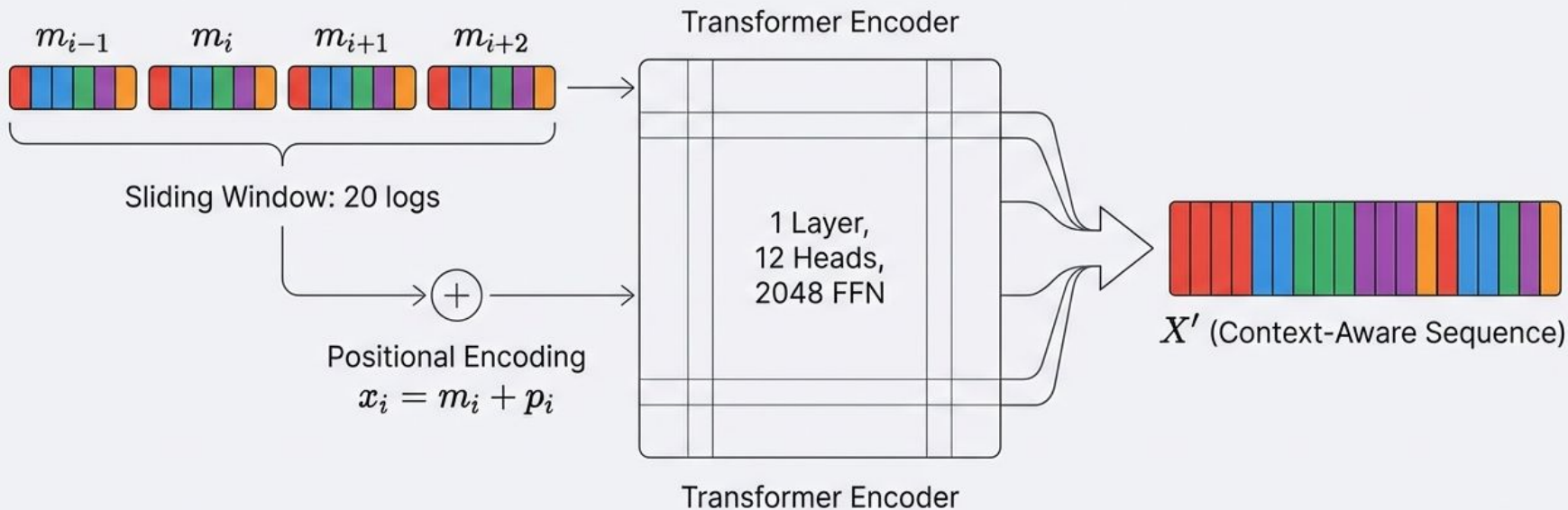
Role-Aware Pooling



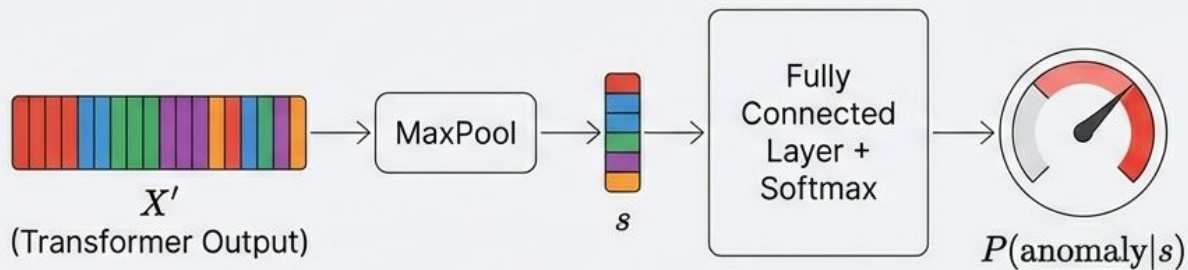
3840 Dimensions (5 Roles x 768). Preserves Hierarchical Structure.

$$m = [m_{\text{state}}; m_{\text{entity}}; m_{\text{action}}; m_{\text{quantity}}; m_{\text{location}}]$$

Modeling Temporal Dependencies



From Sequence to Anomaly Detection



Training Recipe

Loss Function: Cross-Entropy

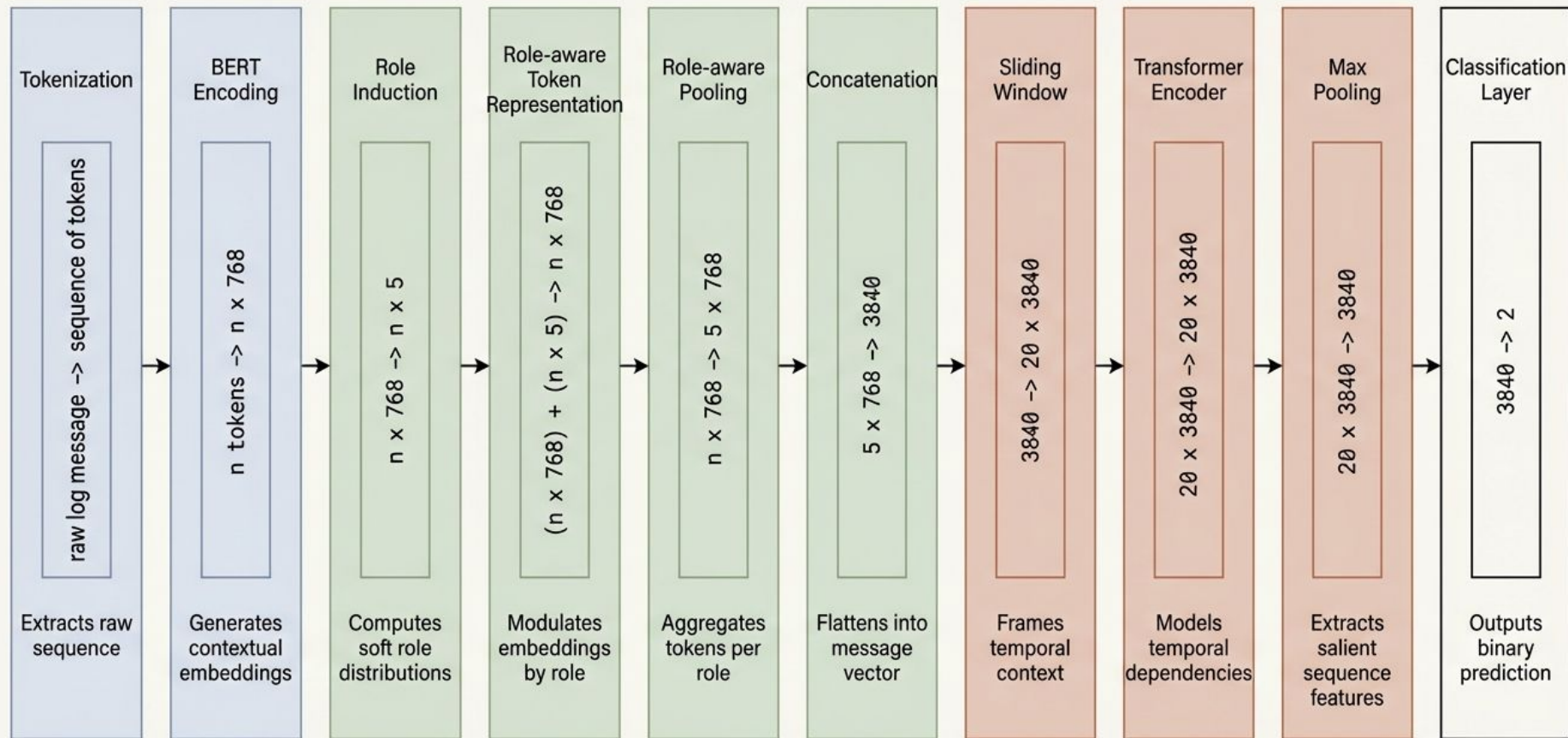
Optimizer: AdamW ($lr = 3 \times 10^{-4}$)

Batch Size: 64

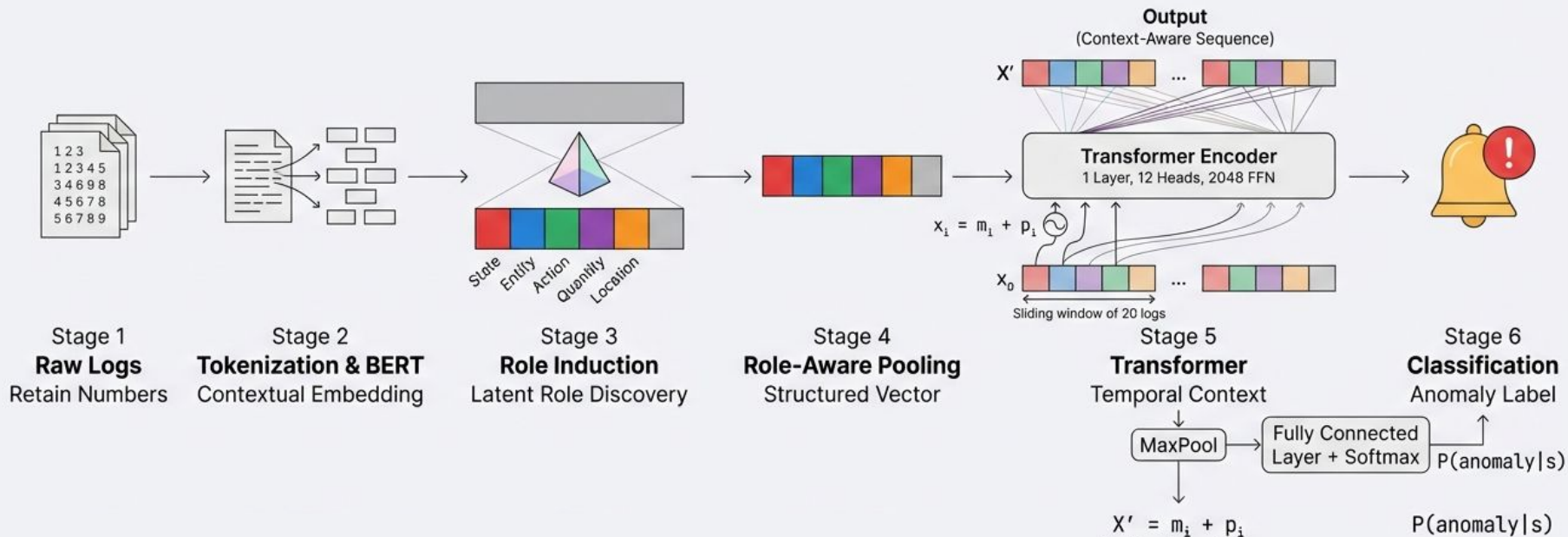
Dropout: 0.1

Early Stopping: 5 epochs (Max 20)

Dimensional Flow Through Pipeline



The Complete Architecture: From Raw Log to Decision



Probabilistic Mapping

Every row represents a soft probability distribution across the five structural dimensions.

Token	State	Entity	Action	Quantity	Location	Dominant Role
CE	0.62	0.12	0.08	0.10	0.08	State
sym	0.10	0.18	0.09	0.55	0.08	Quantity
14	0.05	0.06	0.05	0.80	0.04	Quantity
location	0.08	0.10	0.07	0.05	0.70	Location
R02-M1-N1	0.05	0.12	0.06	0.08	0.69	Location
detected	0.14	0.07	0.72	0.04	0.03	Action
error	0.85	0.06	0.04	0.03	0.02	State
node	0.08	0.78	0.06	0.04	0.04	Entity
core	0.07	0.80	0.05	0.05	0.03	Entity

Emergent Classification

Notice how the highest probability in the distribution explicitly dictates the Dominant Role on the far right.

Syntax STATE ENTITY ACTION QUANTITY LOCATION

Precision Through Role Awareness

Old Paradigm	New Paradigm
Keyword matching & Averaged vectors.	Deep understanding of functional roles.
Blind to numeric anomalies.	Sensitive to counts (Purple), IDs (Blue), and complex states (Red).
Treats data as a 'soup'.	Treats data as a 'structured signal' (Action: Green).

- **Numeric Fidelity:** We treat numbers as vocabulary, not noise.

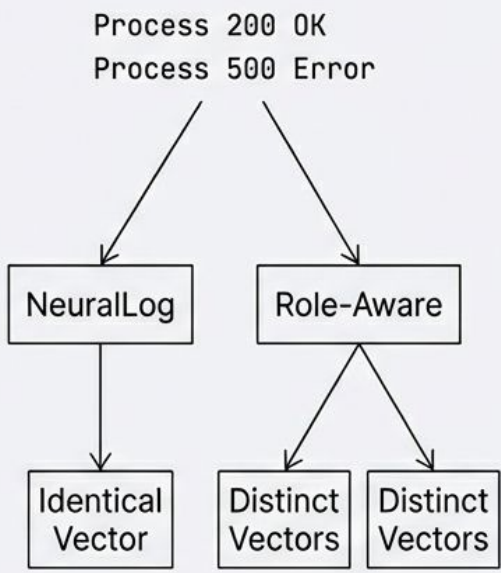
- **Latent Structure:** We discover functional roles without manual effort.

- **Structured Signal:** We allow the model to 'read' the log message with the same nuance as a human site reliability engineer.

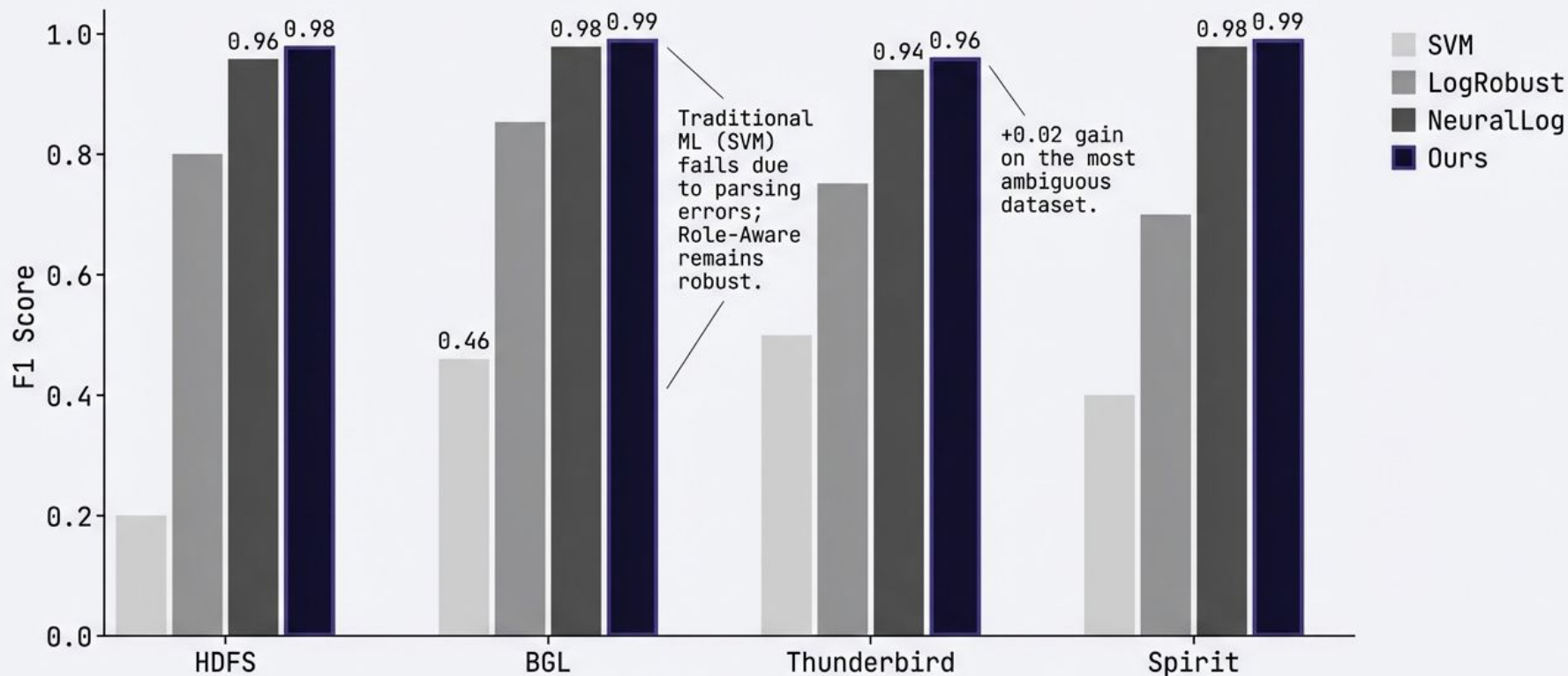
The Current Standard Has a Structural Blind Spot

Comparing NeuralLog (Baseline) vs. Role-Aware Representation

Feature	NeuralLog (Baseline)	Role-Aware (Ours)
Token Treatment	Removes Numbers (Blind to Error Codes)	Preserves Numbers (Context Aware)
Message Pooling	Uniform Average (Smeared Meaning)	Role-Separated (distinct semantic channels)
Structure Modeling	Implicit / Flat	Explicit / Structured
Vector Dimension	Limited (768 Dimensions)	Enhanced (3840 Dimensions)



Dominating Benchmarks with SOTA F1 Scores



The Thunderbird Breakthrough

$$\Delta_{\text{err}}^{\text{Thunderbird}} = \frac{1 - 0.98}{1 - 0.96} = 0.50$$

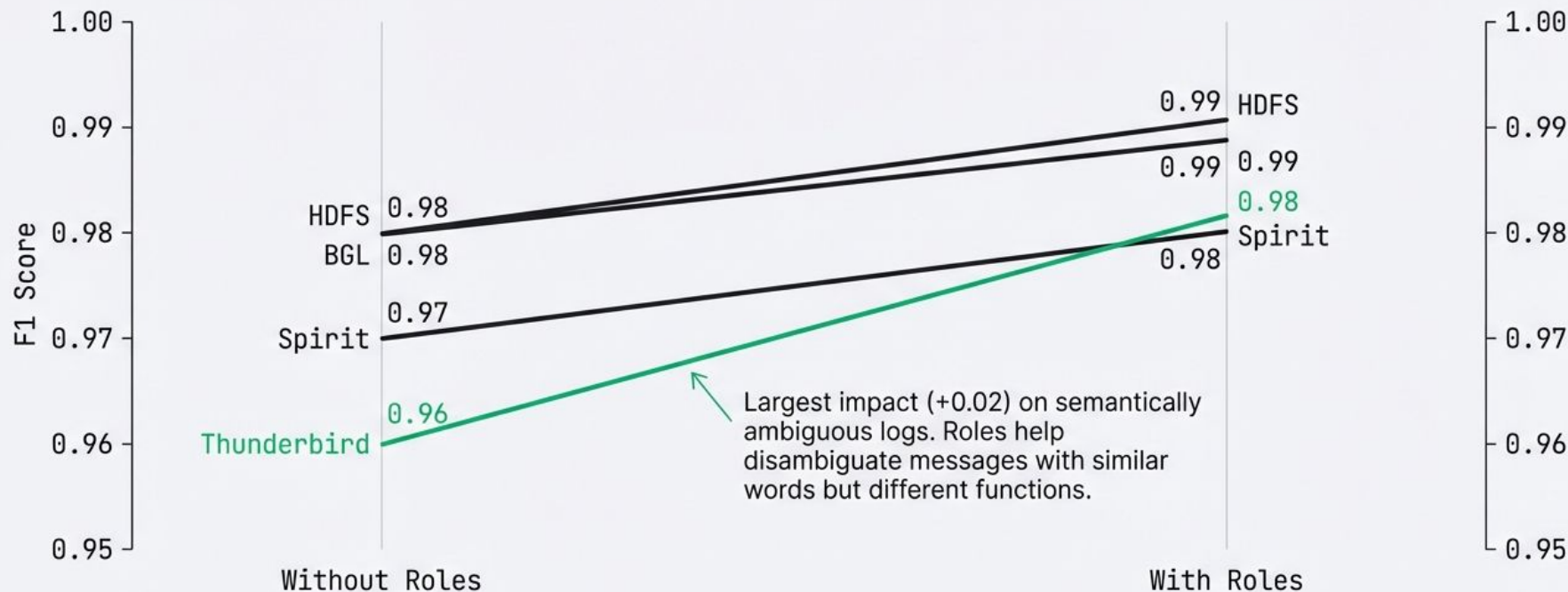
Role-Aware F1-Score

Previous SOTA F1-Score (NeuralLog)

50% relative error reduction

Halving the total volume of undetected anomalies and false alarms on a 10-million message dataset.

Ablation Study: Do Roles Actually Help?



Conclusion: Removing role modeling consistently degrades performance across all datasets.

Ablation Study: The Criticality of Numeric Tokens

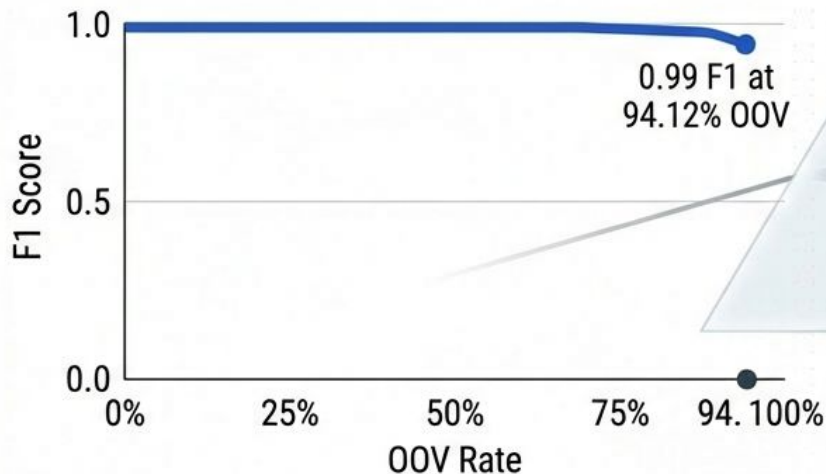
NeuralLog View		Role-Aware View
Process [NUM] Error	+0.03 Gain 	Process 500 Error
0.96 F1 (BGL)		0.99 F1 (BGL)

Retaining numbers is crucial for BGL and Thunderbird datasets. These systems rely heavily on specific error codes (500 vs 200) to indicate failure. NeuralLog strips these, losing context. We model them as "Quantity" roles.

Dataset: BGL	Numbers Removed: 0.96	Numbers Retained: 0.99
Dataset: Thunderbird	Numbers Removed: 0.95	Numbers Retained: 0.98

Robustness & Efficiency

The OOV Challenge



Maintains 0.99 F1 score even with 94.12% Out-of-Vocabulary rate on BGL.

Computational Cost

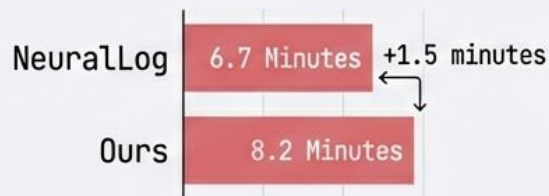
Task	NeuralLog	Ours	Delta
Preprocessing	14.3m	15.8m	+1.5m
Inference	3.1ms	3.8ms	+0.7ms

Negligible overhead compared to parsing-based methods, with significantly higher stability.

Limitations & Future Scope

Complexity

The Role Induction Network adds parameters, making the model slightly heavier than NeuralLog to train (+1.5 minutes).



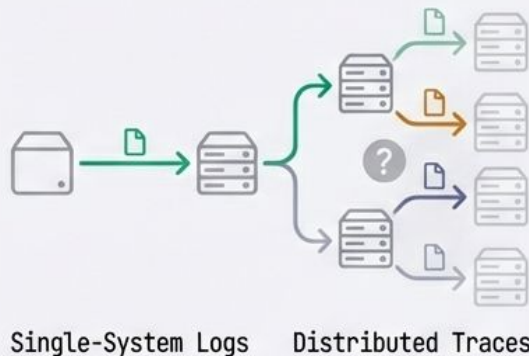
Static Roles

We utilize a fixed set of 5 roles. While empirically effective for these datasets, the optimal number might vary by domain.



System Scope

Evaluation performed on single-system logs. Distributed traces with cross-system dependencies have not yet been evaluated.



Role-Aware Log Representations for Enhanced Anomaly Detection

CONTRIBUTION OVERVIEW	■ Introduces an unsupervised framework modeling token-level functional roles in raw logs ■ Replaces uniform token processing with structure-preserving representation mechanisms ■ Achieves state-of-the-art F1-scores (0.98–0.99) across four benchmark datasets
METHOD SUMMARY	■ Role extraction from logs: Induces functional categories via an unsupervised neural network ■ Representation construction: Aggregates distinct embeddings using role-aware pooling ■ Anomaly detection improvement: Applies transformer encoders to capture temporal dependencies
KEY ADVANTAGES	■ Bypasses rigid template parsing frameworks and their subsequent error propagation ■ Preserves critical numeric telemetry previously discarded by traditional methods ■ Provides explicit predictive interpretability through functional role attribution

Note: Accepted at Lecture Notes in Networks and Systems, Springer Nature (CVR 2026, SCOPUS-indexed)

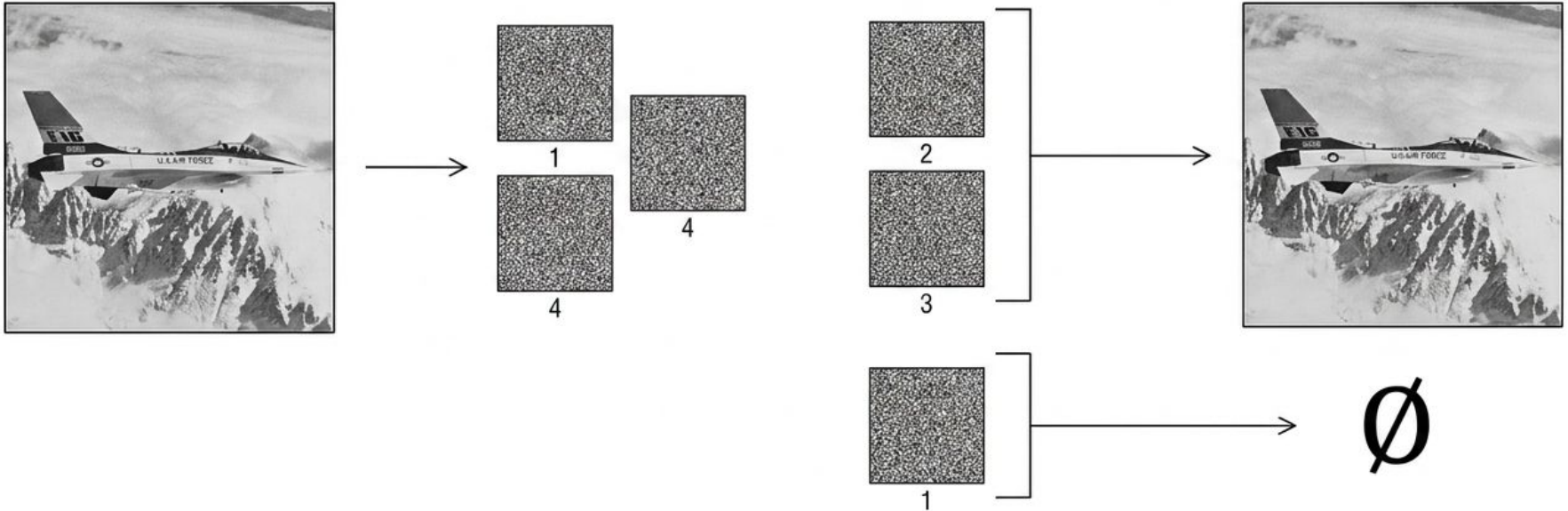
Optimizing Secret Image Sharing

Balancing Storage Efficiency,
Security, and Perfect Reconstruction



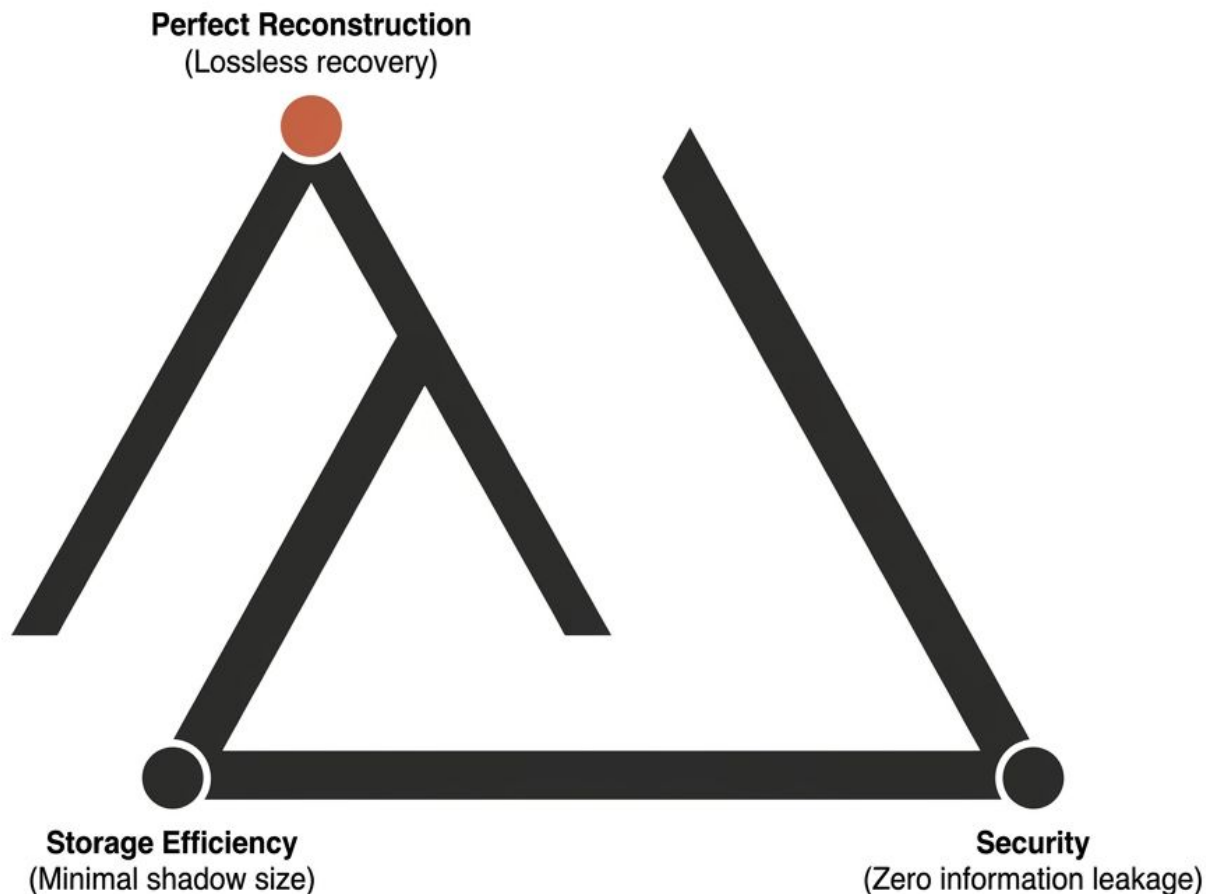
The (t,n) Threshold Cryptographic Mechanism

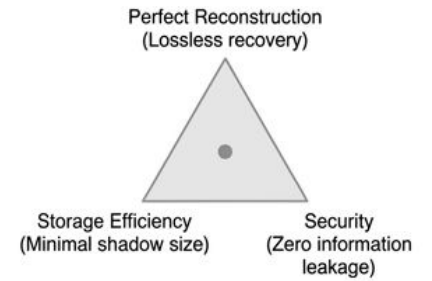
A threshold t (e.g., 2) of total shares n (e.g., 4) is required for reconstruction. Fewer than t shares reveal absolutely zero structural information.



The Optimization Triad

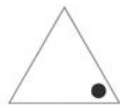
Optimizing decentralized image storage is a delicate balancing act. Every variable altered to shrink file size mathematically threatens either the cryptographic security of the shares or the pixel-perfect fidelity of the reconstruction. The goal is a methodology that satisfies all three vertices simultaneously.





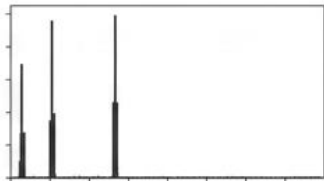
Reduces shadow image size by ~40% compared to legacy Thien & Lin methods.

Hypothesis 1: Removal of Leading Zeros (RLZ)



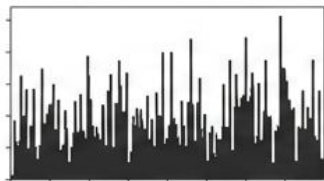
Panel A: Standard 8-bit

000000100



Panel B: RLZ Preprocessed

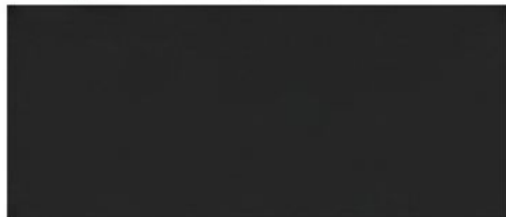
100



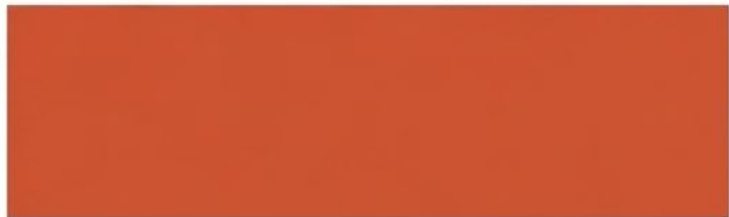
Stripping leading zeros logically reduced the initial bitstream. However, it exponentially increased unique pixel values. This forced the Huffman dictionary to maximum capacity (509+ entries), yielding a paradoxical increase in final shadow image size.

The Huffman Paradox

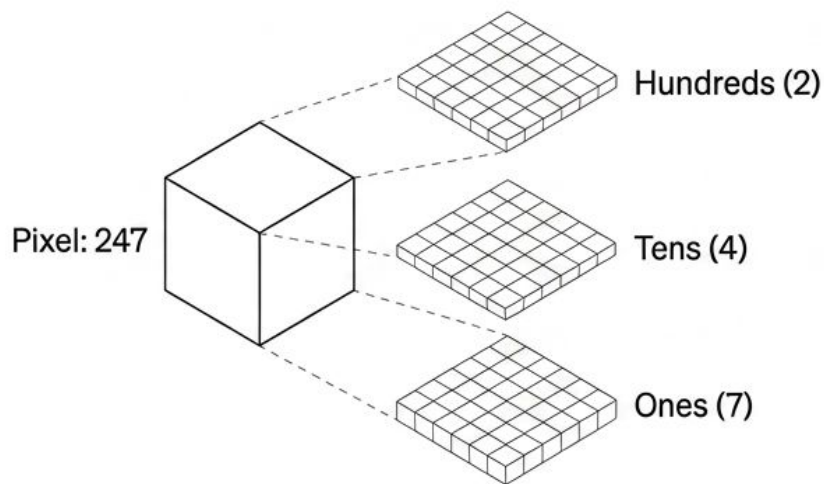
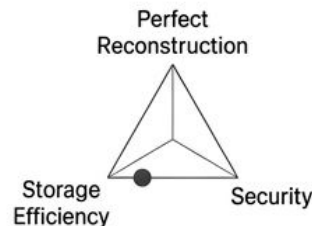
Compressed Preprocessed Size: 211 KB to 250 KB



Resulting Shadow Image Size (Bloat)



Hypothesis 2: Digit Decomposition (DDP)

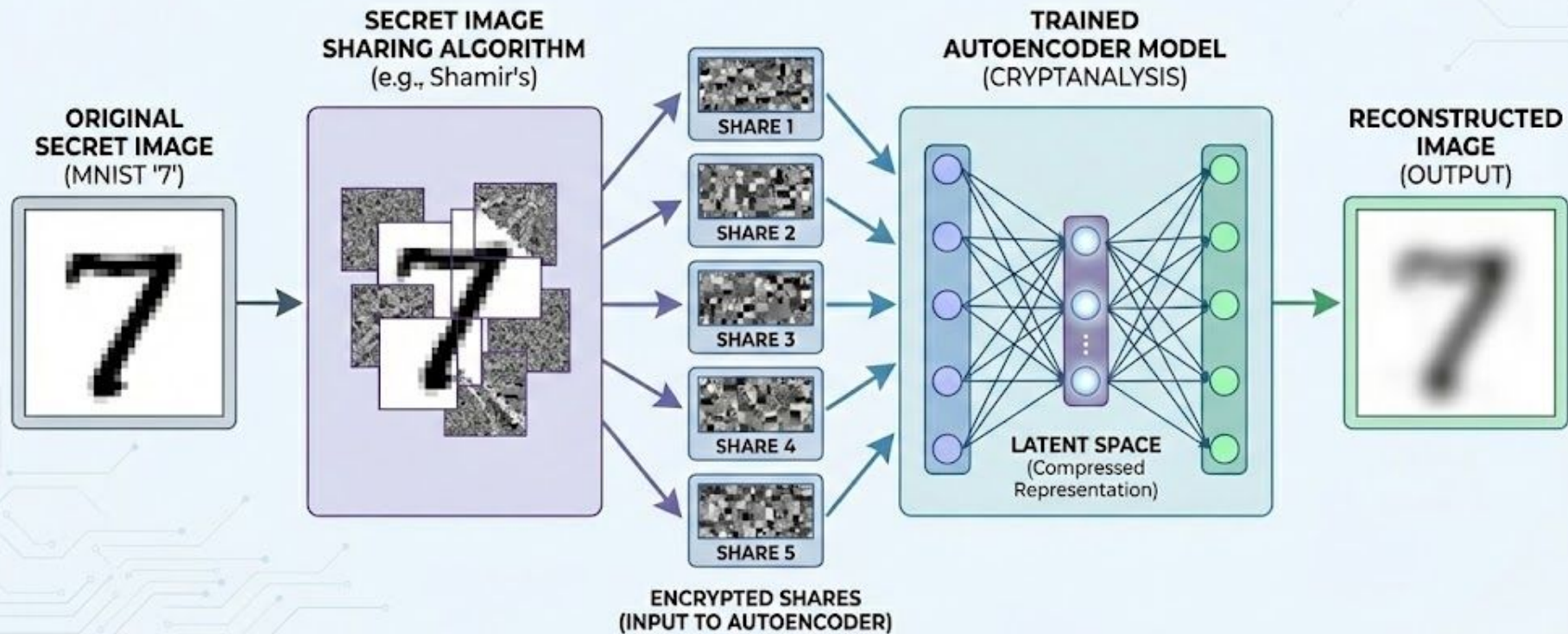


- **Preprocessed Size Comparison:**
 - DDP: 191 KB - 212 KB
 - RLZ: 211 KB - 250 KB
- **Final Storage Outcome:**
 - Both methods maximized Huffman entries (DDP: 511, RLZ: 509+).
 - Result: No final storage improvement for shadow images.

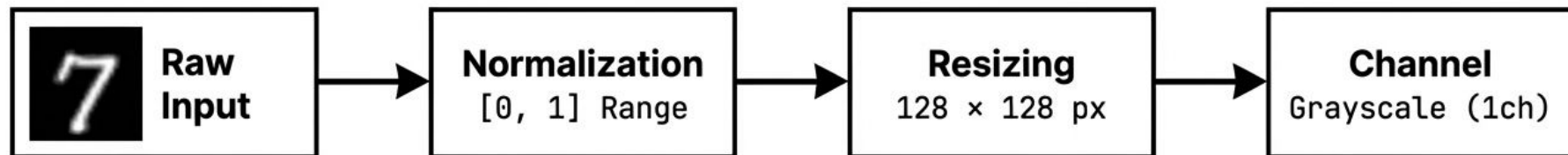
Both DDP and RLZ successfully reduced the initial bitstream size. However, the increased complexity of unique values forced the Huffman dictionary to its limit in both cases, preventing any final reduction in shadow image size.

CONCLUSION: BITSTREAM MANIPULATION PRIOR TO HUFFMAN CODING IS A MATHEMATICAL DEAD END.

AUTOENCODER BASED CRYPTANALYSIS ON SECRET IMAGE SHARING SCHEMES



DATA PROCESSING PIPELINE



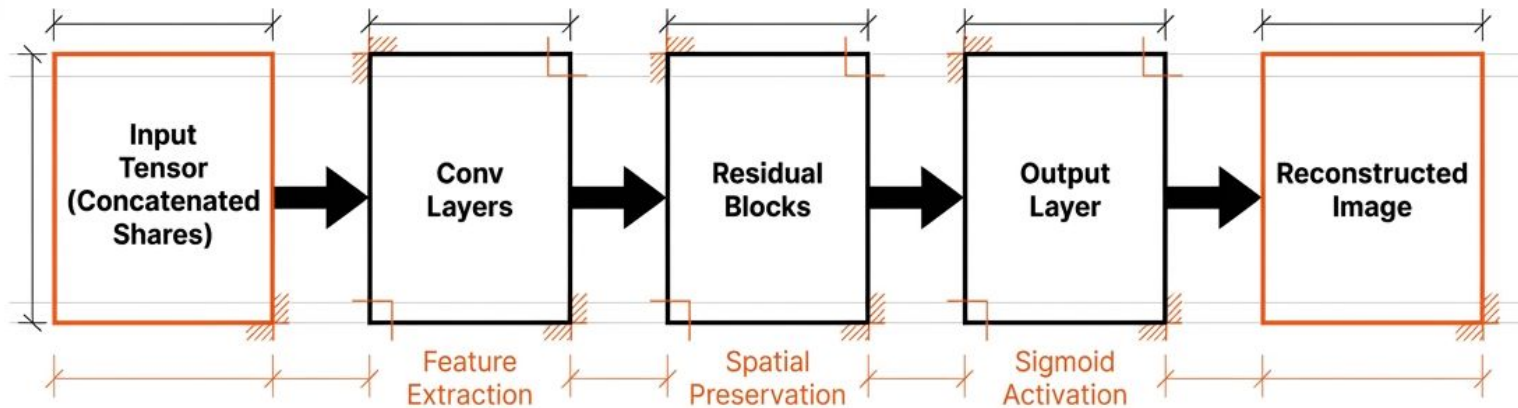
DYNAMIC TRAINING PROTOCOL



On-the-fly Share Generation

Shares are generated dynamically during every training epoch. The network never sees the same noise pattern twice for the same image.

Goal: Prevent memorization of fixed pairs. The model must learn the reconstruction function, not the examples.

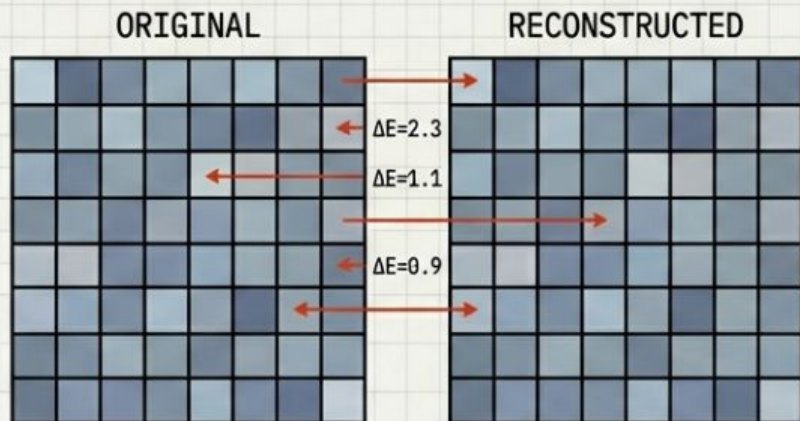


TECHNICAL SPECS

- **Optimizer:**
Adam
- **Loss Function:**
Binary Cross-Entropy
- **Regularization:**
Batch Norm + Dropout
- **Training Duration:**
20 Epochs

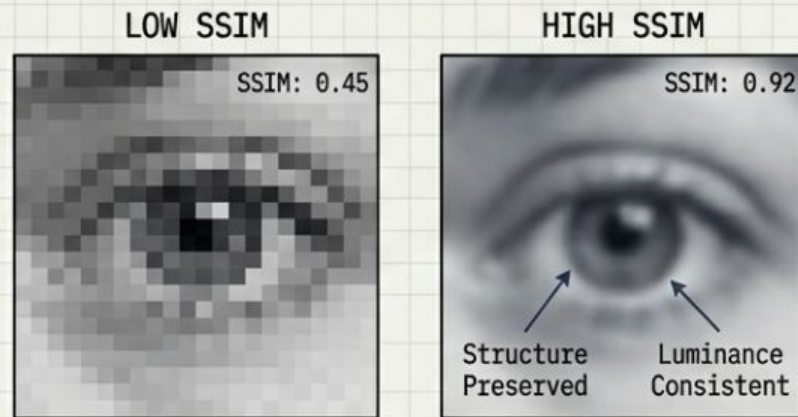
Defining Quality: Math vs. Perception

PSNR (Peak Signal-to-Noise Ratio)



Measures absolute pixel-level error.
Mathematical precision.

SSIM (Structural Similarity Index)

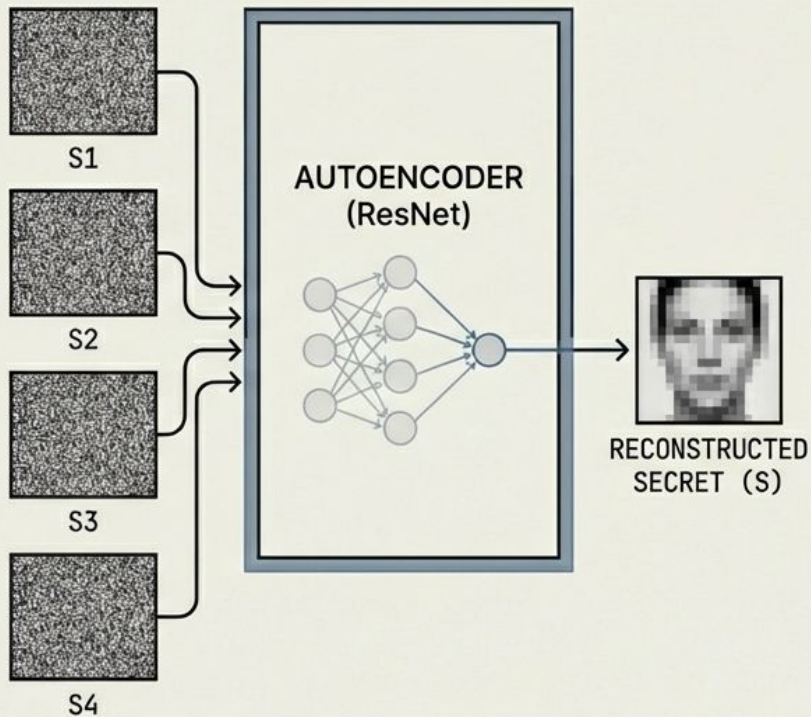


Measures perceptual quality (Luminance, Contrast, Structure). How "real" it looks to a human.

- Modern reconstruction requires optimizing both. We cannot rely on 'human visual' decryption alone.

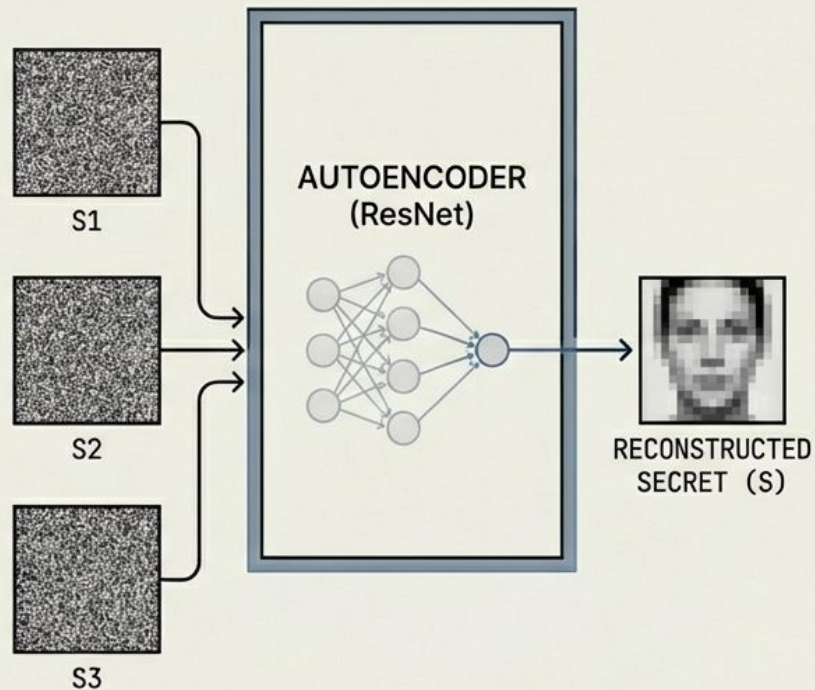
Experimental Methodology: Decoder Configurations

CONFIGURATION 1: FULL KNOWLEDGE



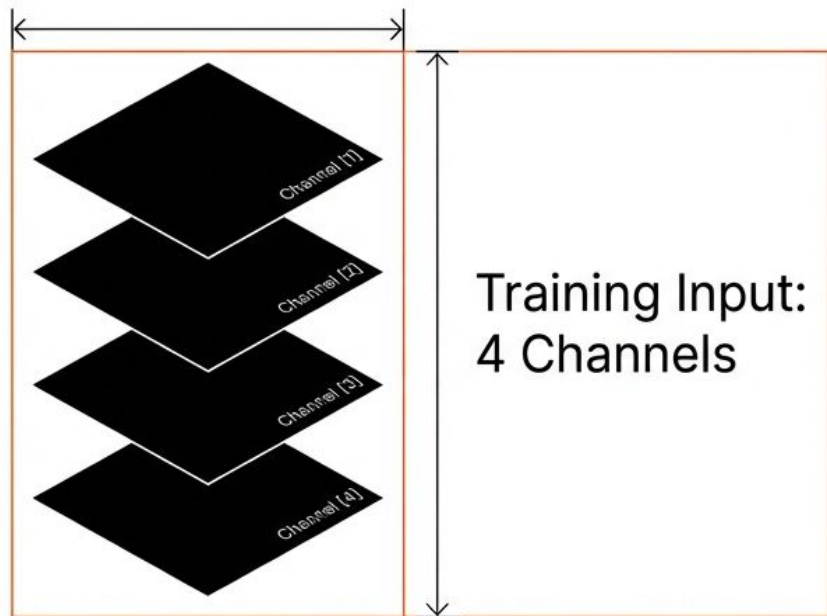
Objective: Test if the model can learn the XOR function.
Trained on ALL 4 shares.

CONFIGURATION 2: PARTIAL KNOWLEDGE



Objective: Model a realistic attacker with limited access.
Trained on ONLY 3 shares to find statistical correlations.

Configuration 1: Full Exposure Training



INFERENCE STRESS TEST

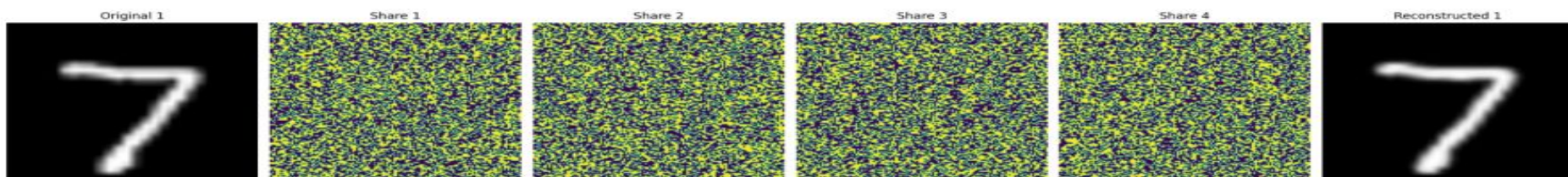
■ ■ ■ ■ 4 Shares

■ ■ ■ □ 3 Shares

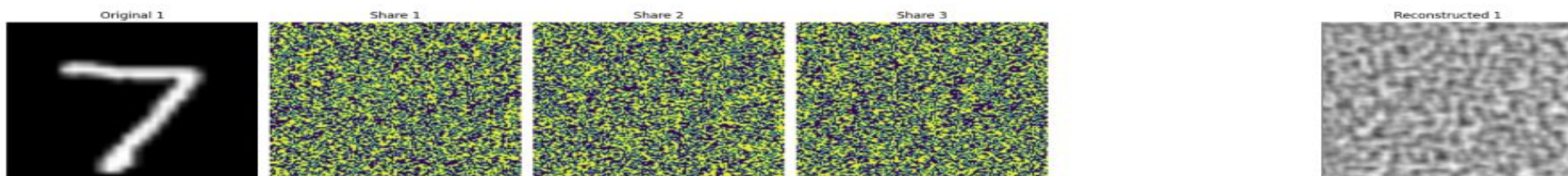
■ ■ □ □ 2 Shares

Mapping Function: $f(\theta): R(128 \times 128 \times 4) \rightarrow R(128 \times 128)$

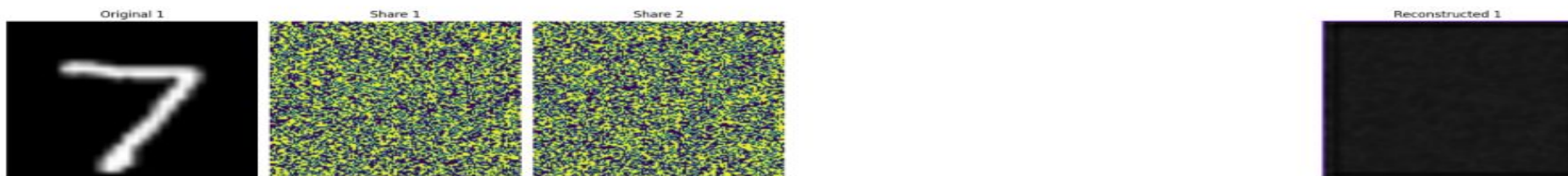
Condition: Decoder sees all shares during training, but faces data loss during the attack phase.



(a) All 4 Shares



(b) 1 Share Missing

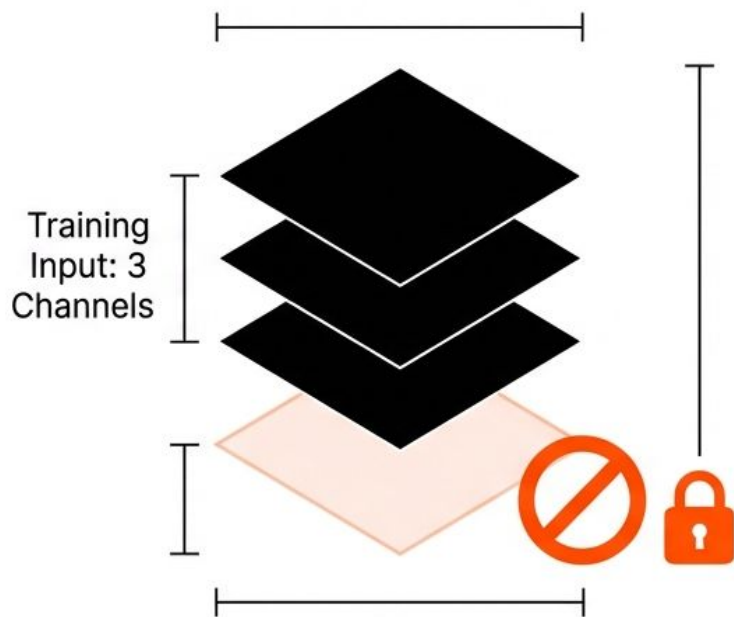


(c) 2 Shares Missing



(d) 3 Shares Missing

Configuration 2: The Constrained Adversary



Scenario:

The decoder is trained AND evaluated using only three consecutive shares.

Note:

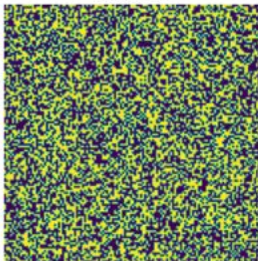
No wrap-around permitted.

Objective: Can the model approximate the secret despite being mathematically locked out of the solution?

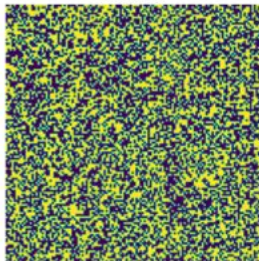
Original 1



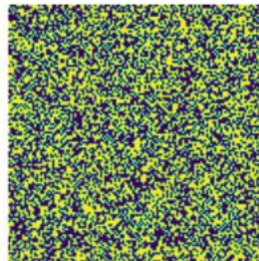
Share 1



Share 2



Share 3



Reconstructed 1

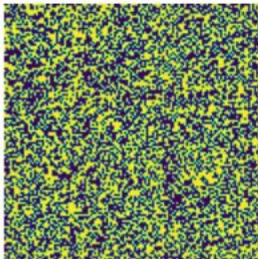


(e) Shares $\{1,2,3\}$

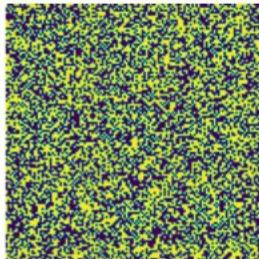
Original 1



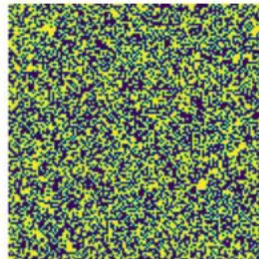
Share 2



Share 3



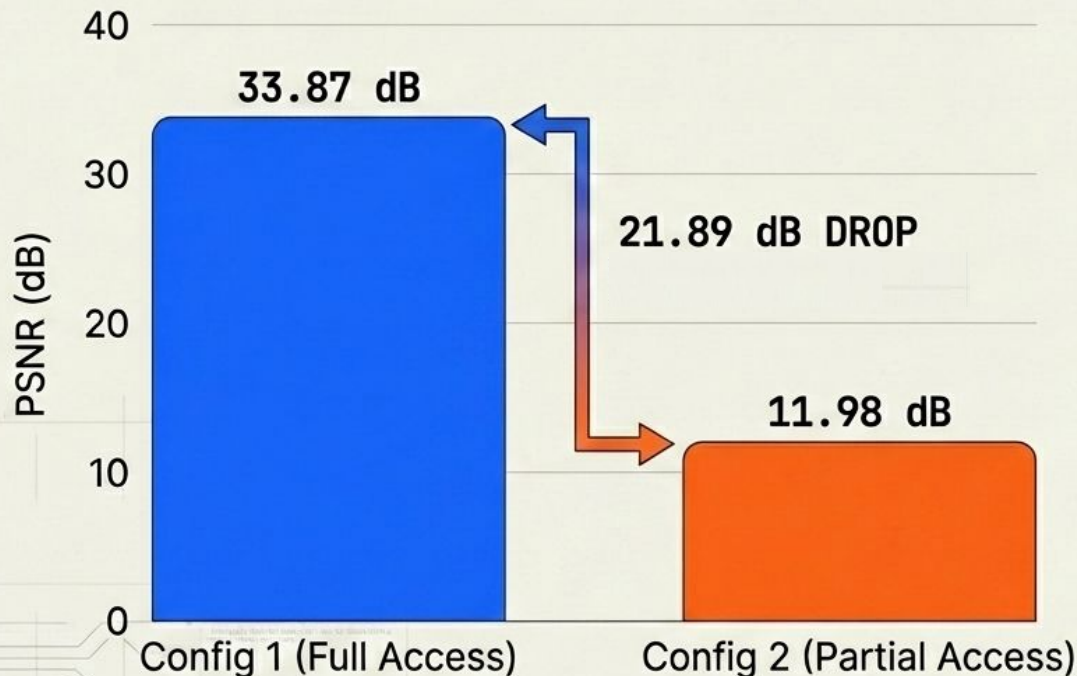
Share 4



Reconstructed 1



Quantitative failure confirms lack of correlation



ANALYSIS: A drop of ~20 dB represents a total collapse in signal quality.

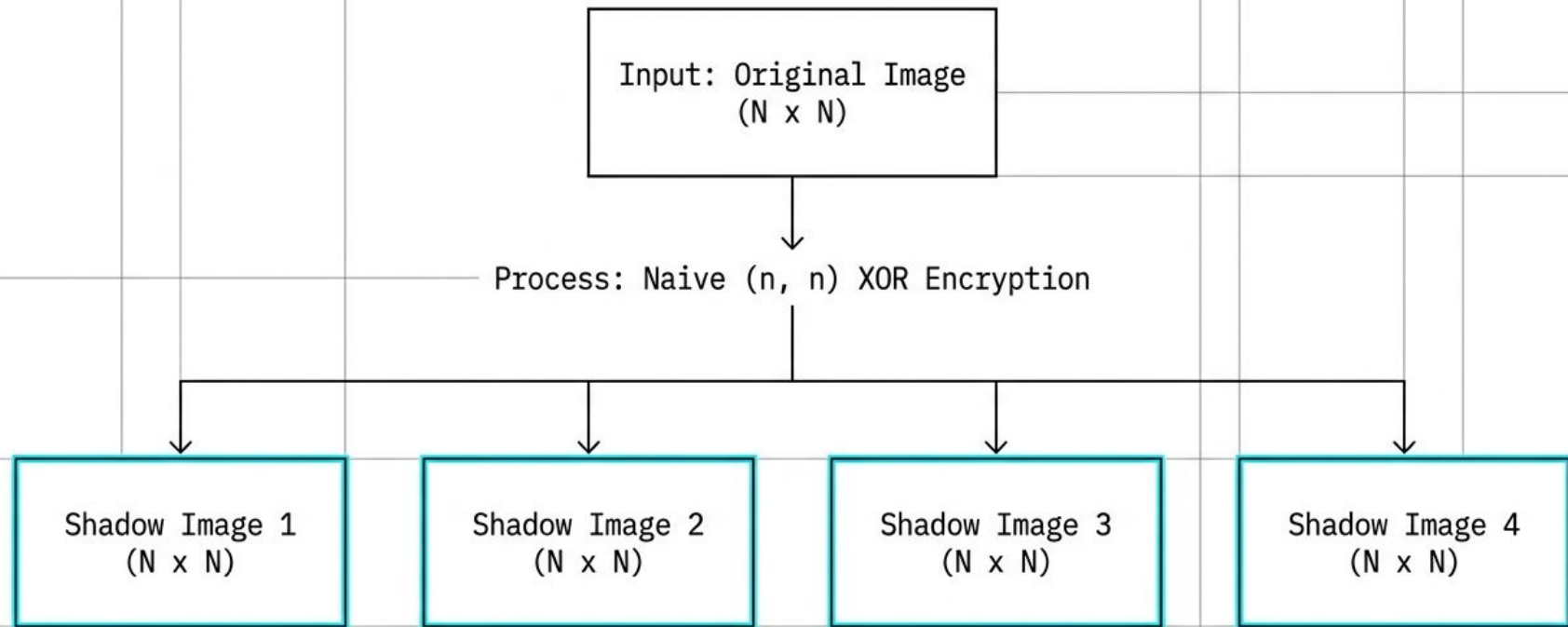
The SSIM (structural similarity) drops to ~0.20, meaning the output is mathematically unrelated to the input.

Even when explicitly trained to find a link, the AI **cannot find one** because it does not exist.

This paper was submitted at 4th International Conference on Networks and Cryptology (NETCRYPT 2026).

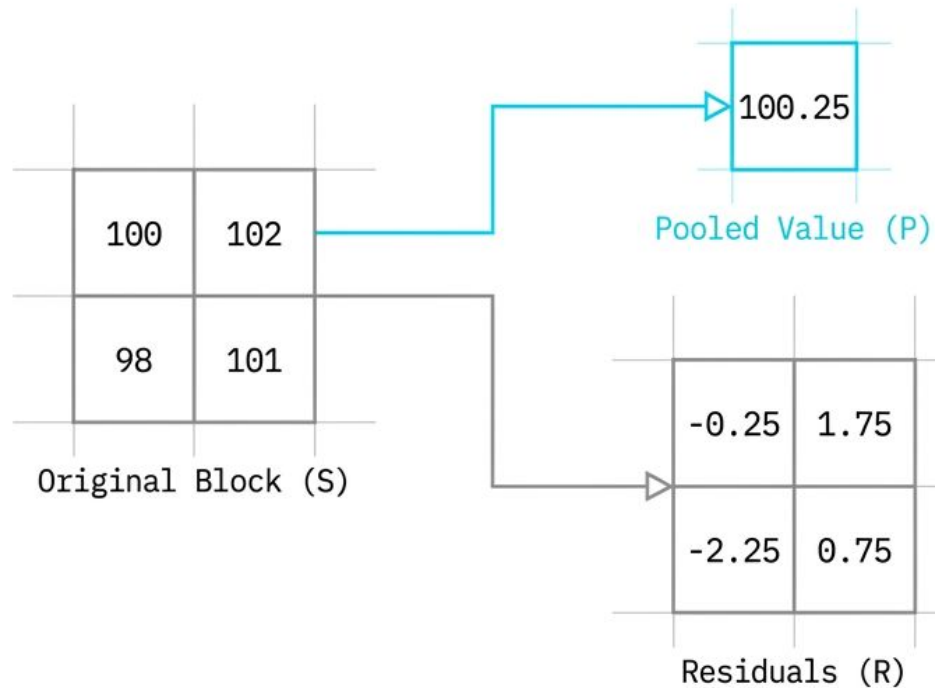
11	16		
11	11	PoolLiteSIS: Dimensional Reduction in Secret Image Sharing Breaking the $n \times n$ size barrier using average pooling and cyclic XOR cryptography.	12 
	12		16

Conventional XOR Sharing



While Boolean XOR operations provide lightweight, perfect cryptography, they fail to compress. Generating 4 shares of a 256KB image requires storing 1,024KB. The system scales its own redundancy.

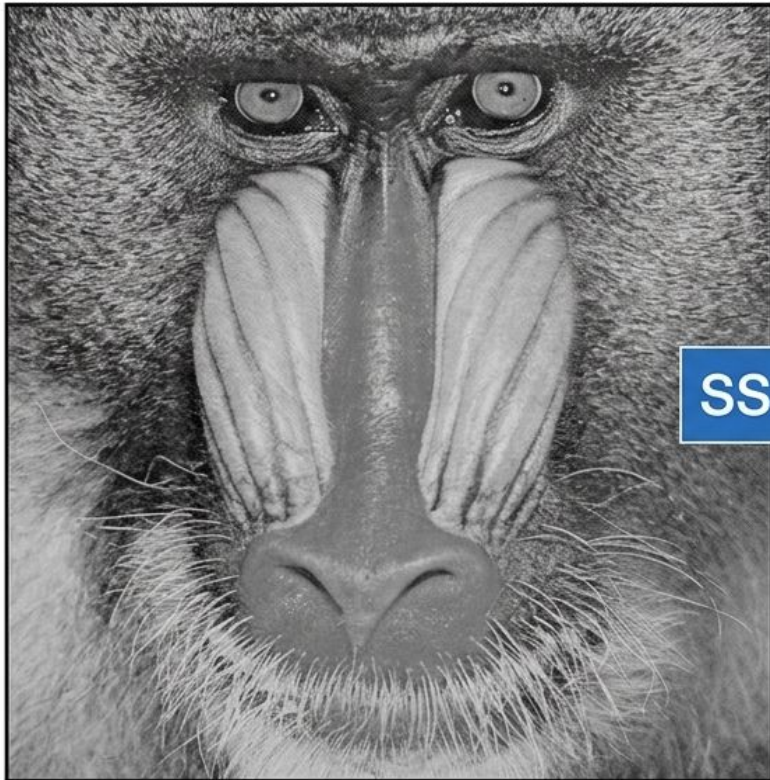
Phase 1: Average Pooling & Dimensional Reduction



Spatial dimensions reduced by 75% ($K=2$, stride $s=2$).

$$R(x, y) = S(x, y) - P\left(\left\lfloor \frac{x}{s} \right\rfloor, \left\lfloor \frac{y}{s} \right\rfloor\right)$$

The Security Leak: Analyzing the Residuals

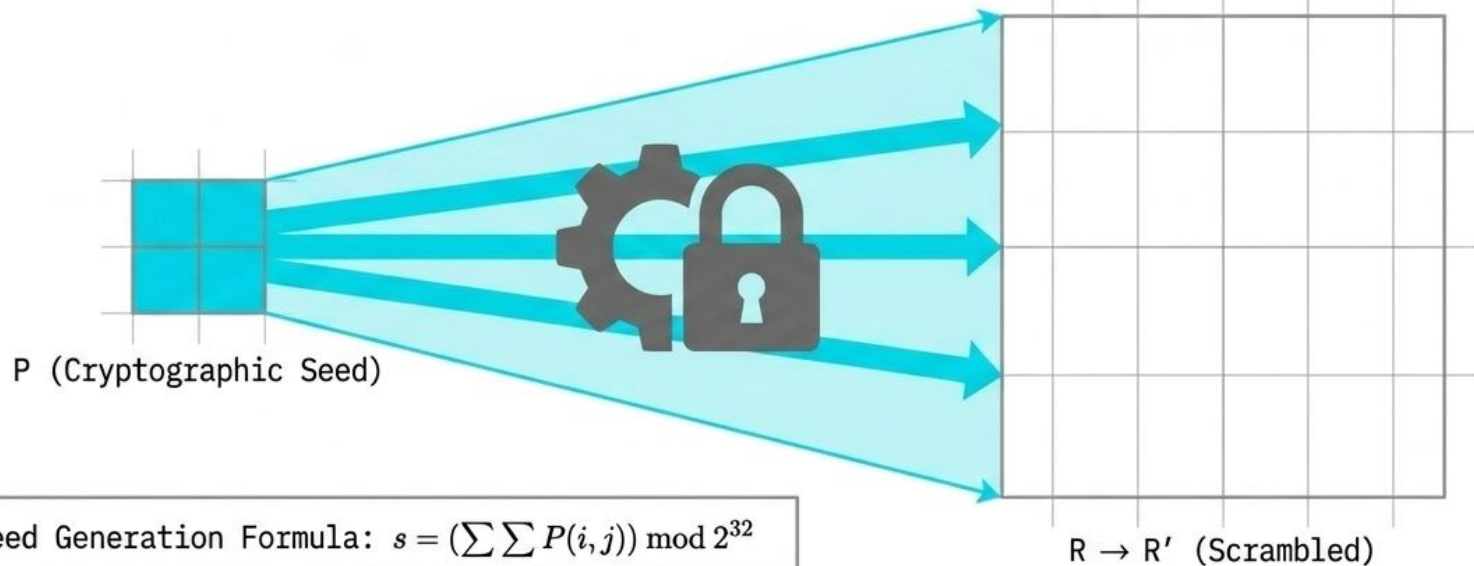


SSIM = 0.85



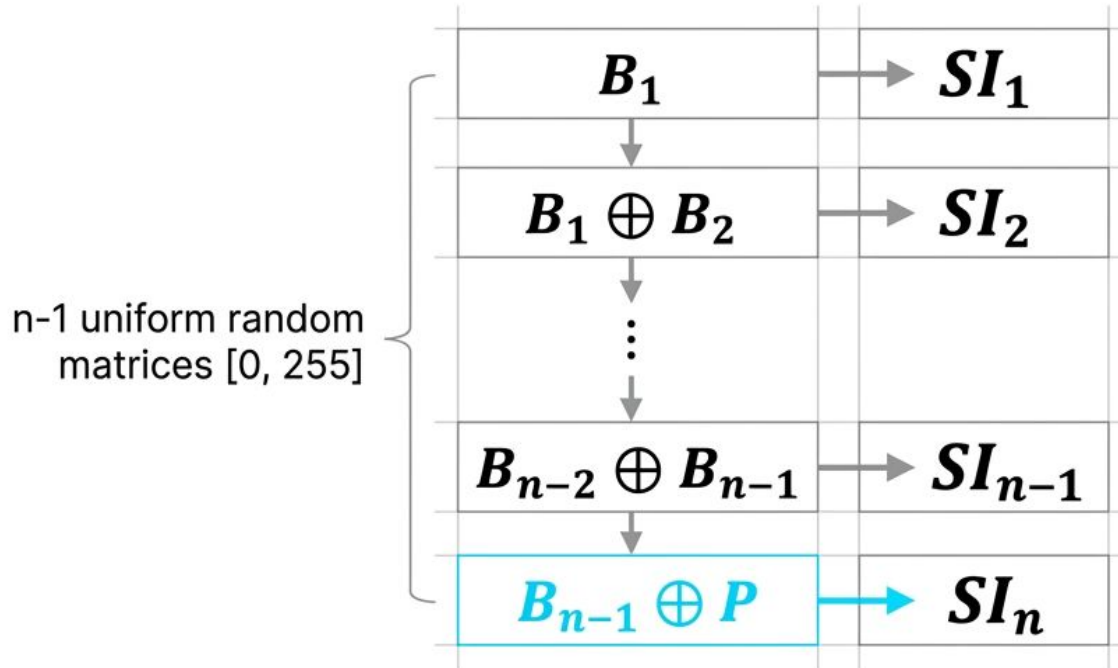
While spatial pooling solved the storage bottleneck, the residual map retained significant structural correlation to the original image. Publishing the residuals as a public map compromised cryptographic secrecy, rendering the method invalid.

Phase 2: Residual Permutation (Key-Free Security)



A major security vulnerability occurs if unpermuted residuals mimic the original image. By using the pixel sum of the pooled image P as the cryptographic seed s , the residual map is shuffled deterministically. No external keys are exchanged; the key is mathematically locked inside the shares.

Phase 3: Cyclic XOR-Based Shadow Generation



The dealer creates $n-1$ uniform random noise matrices. The pooled image P is sequentially embedded into the final shadow image via a cyclic XOR chain.

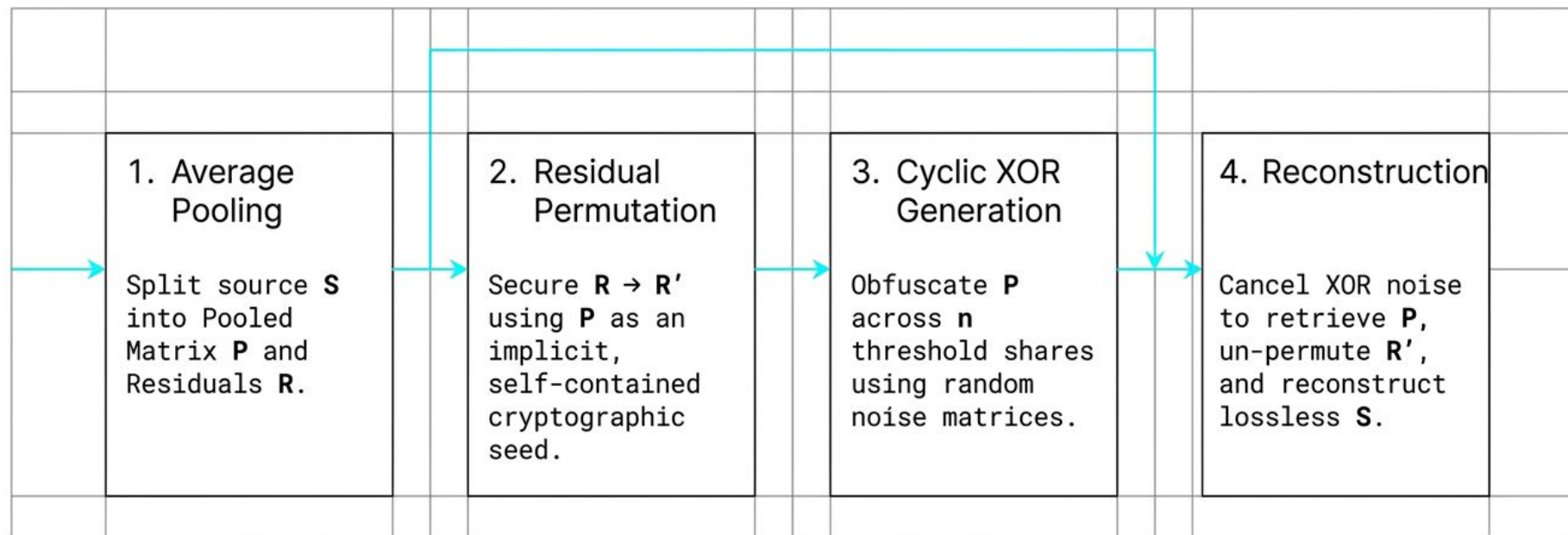
Phase 4: Lossless Reconstruction & XOR Cancellation

$$SI_1 \oplus SI_2 \oplus \dots \oplus SI_n = P$$

$$A \oplus \cancel{B} \oplus \cancel{B} = A$$

Because XOR is its own inverse, stacking all shadow images perfectly cancels out the random noise matrices ($B_1 \dots B_{n-1}$). P is recovered, unlocking the seed to unpermute R . Upsampling P and adding R perfectly restores the original image S .

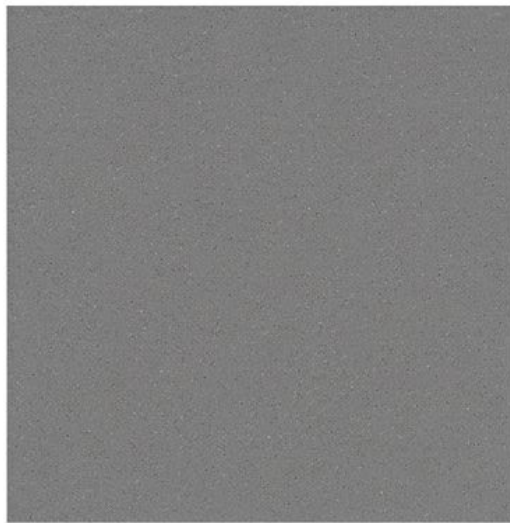
The Algorithmic Pipeline



Visual Verification: Total Structural Destruction

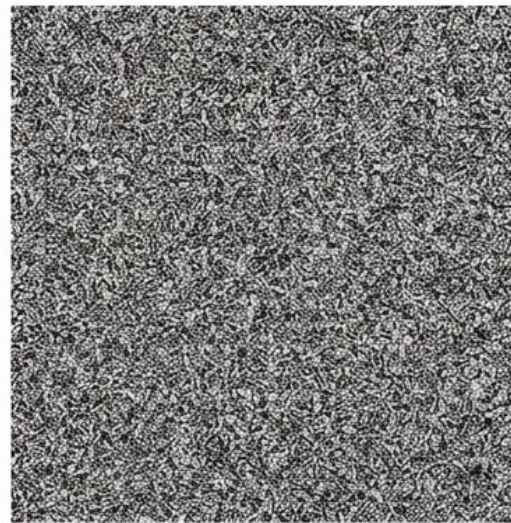


Original Secret Image



Permuted Residuals

SSIM = 0.03. Structural correlation destroyed.



Shadow Image (1 of n)

Shadow size: 256x256 (75% smaller). Appears as pure random noise.

System Performance Dashboard

Reconstruction Fidelity

PSNR = ∞ , SSIM = 1.0000

Tested across standard datasets (Jet, Cameraman, Milk, Baboon). Pixel-perfect copies with zero information loss.

Cryptographic Security

SSIM = 0.01

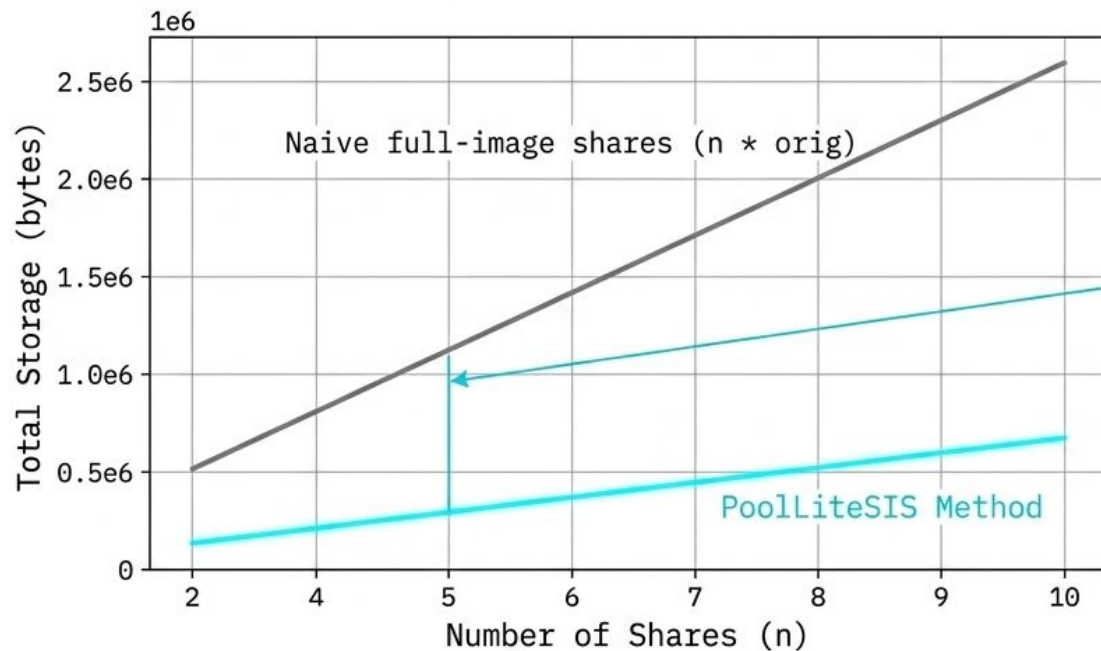
Similarity between individual shadows and pooled image. Evaluates to pure cryptographic noise.

Computational Complexity

Linear $O(mn)$

Operations scale linearly with image size. XOR relies on rapid bitwise parallelism in modern processors.

The Scalability of Dimensional Reduction



At n=4:
Naive requires 1,048,576 bytes.
PoolLiteSIS requires 262,144 bytes.

Because the residual matrix is decoupled from the shares, total storage savings remain **locked at exactly 75%**, regardless of how many shadows are generated.

The PoolLiteSIS

	Naive Methods	PoolLiteSIS
Cryptographic Engine	Bitwise XOR	bitwise XOR
Preprocessing	None	Average Pooling & Residual Permutation
Share Dimensions	$N \times N$ (100% of original)	$N/4 \times N/4$ (25% of original)
Key Management	Requires external shared keys	Key-free (Seed is embedded in data)

A 75% reduction in total storage overhead with zero loss in cryptographic security or reconstruction fidelity.

Model	Lossless	Structure	Share Size
Wang et al. (2007)	Yes	(n, n)	$N \times N$
Chen-Wu (2011/14)	Yes	(n, n)	$N \times N$
Lin et al. (2024)	Yes	(n, n)	$N \times N$
PoolLiteSIS (Proposed)	Yes	(n, n)	$N/4 \times N/4$

1. Storage Efficiency

A baseline 75% reduction in total storage overhead compared to a century of Boolean models.

2. Keyless Autonomy

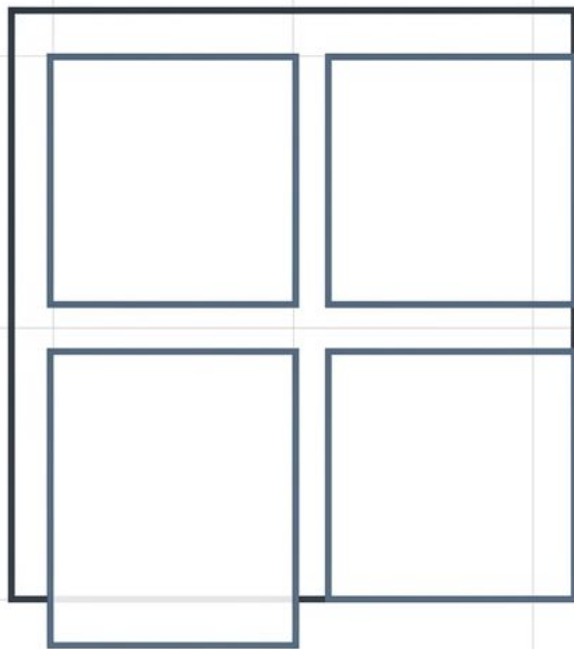
Utilizing the pooled image as an internal cryptographic seed eliminates the vulnerability of external key exchange.

3. Perfect Fidelity

Infinite PSNR and 1.0 SSIM guarantee that dimensional compression does not compromise pixel-perfect data integrity.

Autoencoder-based Secret Image Sharing

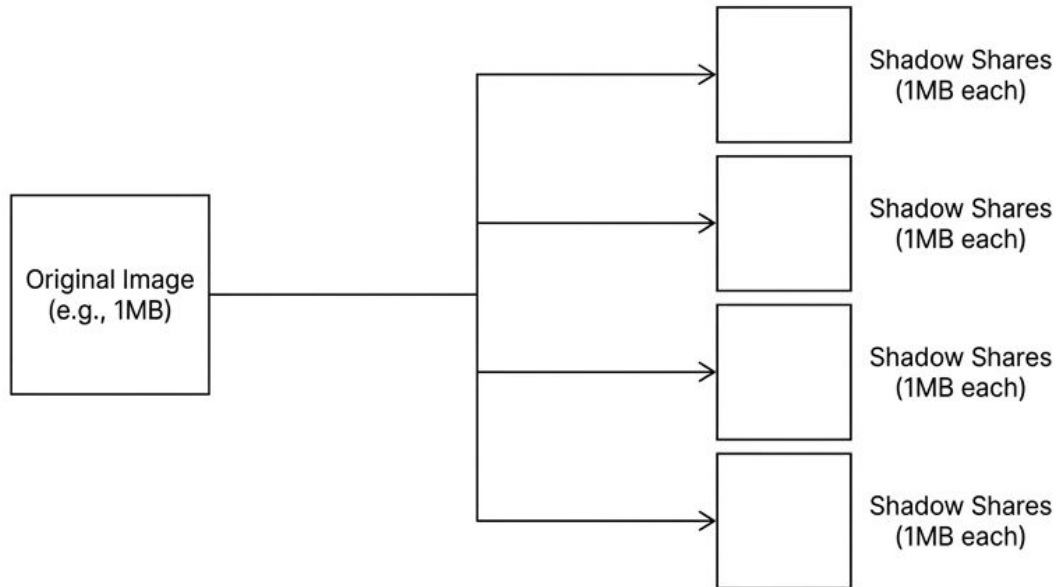
A Deep Learning Approach to
Secure, High-Efficiency Image
Distribution.



The Bottleneck in Traditional Secret Image Sharing

Traditional XOR-based schemes provide theoretically secure cryptographic sharing via Boolean operations. However, they suffer from a critical scaling flaw: massive storage bloat.

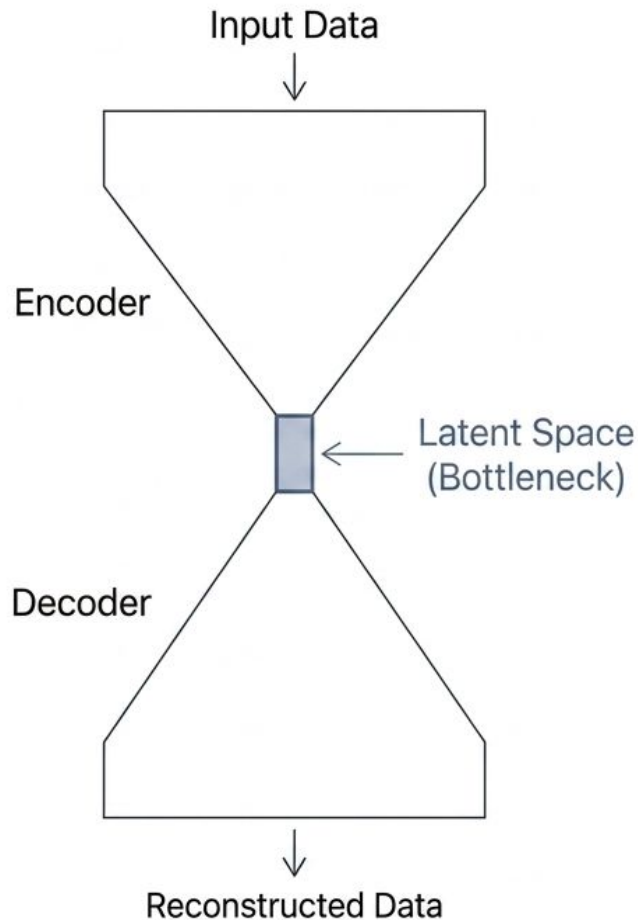
The 400% Storage Overhead Problem



In traditional logical sharing, the shadow shares equal the size of the original image, multiplying storage requirements by the number of shares.

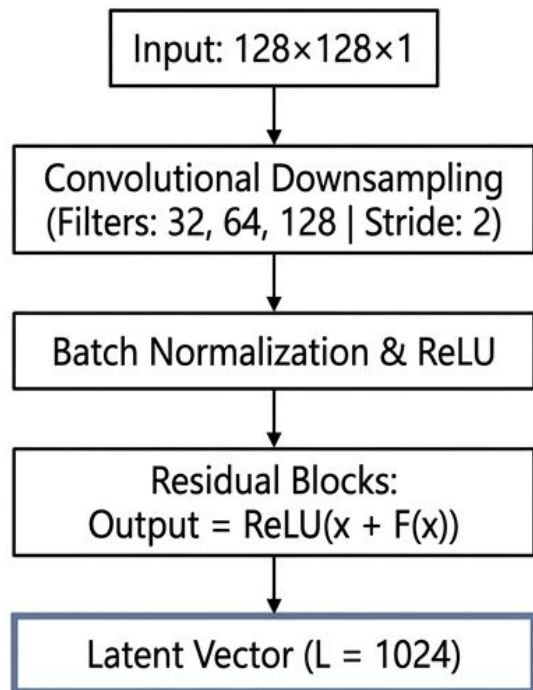
The Paradigm Shift: Learned Compression

Rather than expanding data through polynomial or logical operations, we introduce an undercomplete autoencoder. This deep learning architecture compresses the image into a critical bottleneck—the latent space—before splitting it, drastically reducing redundancy while preserving essential features.



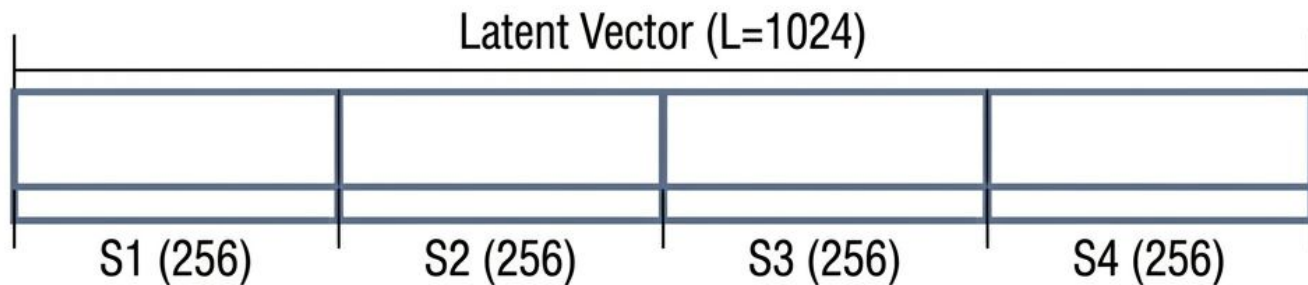
Phase 1: Encoding & Downsampling

The network extracts hierarchical features and stabilizes training via residual blocks before flattening the output into a highly compressed 1024-dimensional bottleneck.



Phase 2: Independent Share Generation

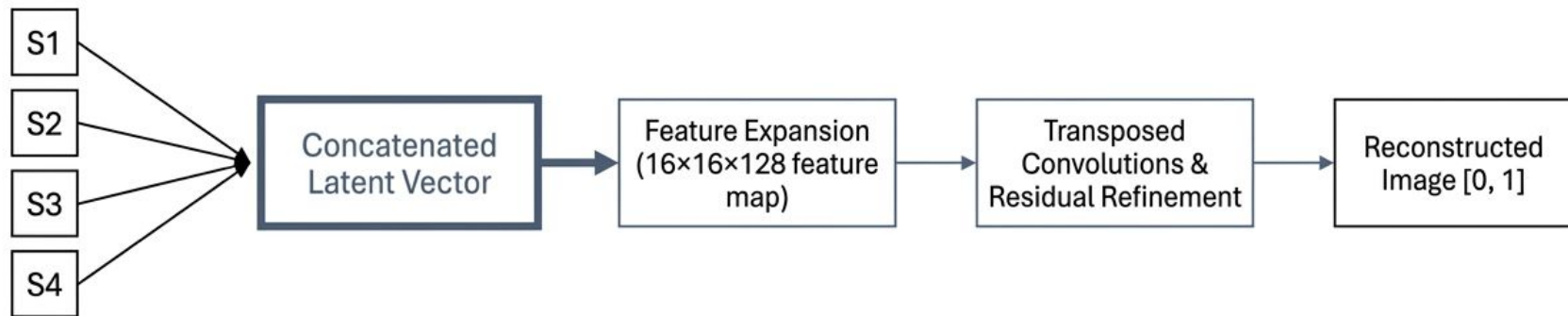
The 1024-dimensional latent vector is strictly divided using a non-trainable splitting operation. Crucially, each share is completely independent.



Fewer than four shares provides zero sufficient information for reconstruction.

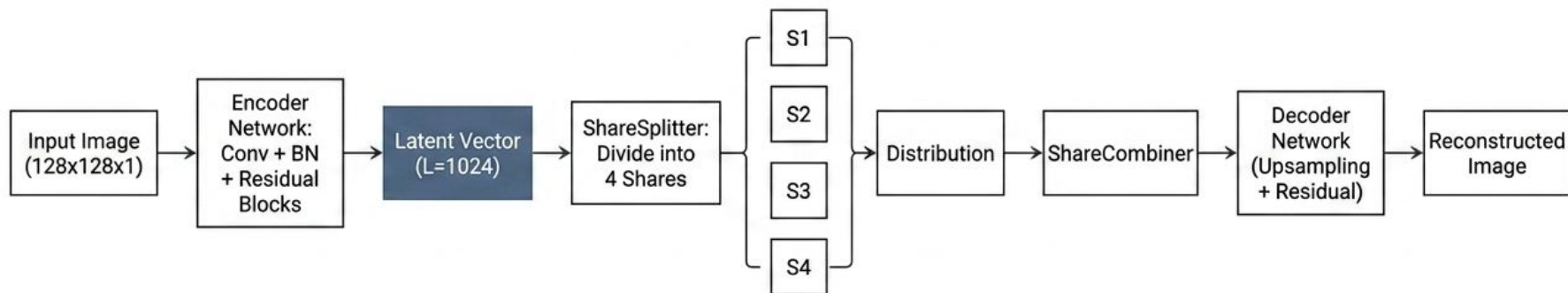
Phase 3: Decoding & Upsampling

Reconstruction requires the exact concatenation of all four shares, which are then expanded back to spatial resolution using transposed convolutions and sigmoid activation.



End-to-End Framework Architecture

The complete pipeline for processing a 128×128 grayscale input into independent shares and reconstructing it from the latent representation.



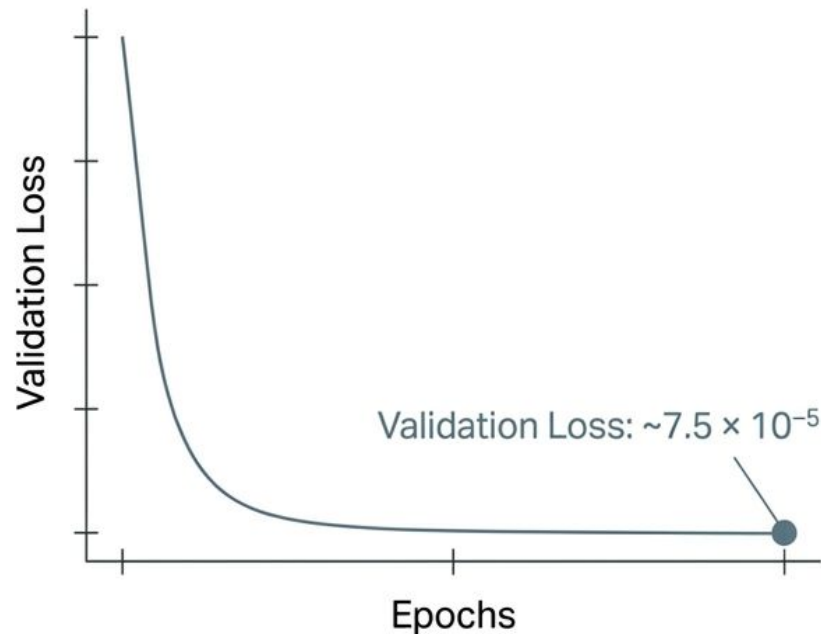
Diagnostic Matrix: Boolean vs. Deep Learning Approaches

Dimension	Traditional XOR (e.g., Wang, Chen)	Proposed Autoencoder
Core Mechanism	Simple Boolean logic	Undercomplete Convolutional Neural Network
Computational Complexity	Low (Resource-constrained friendly)	High (Requires training)
Share Size	1:1 match with Original Image	1/4th of Latent Space (256 values)
Storage Overhead	400% Bloat	Near-Zero
Reconstruction	Strictly Lossless	Lossy (High Fidelity)

Experimental Setup & Training Convergence

The model was evaluated against the standard MNIST dataset to ensure baseline validity and robust feature extraction.

- Dataset: MNIST (Grayscale)
- Split: 60,000 Training / 10,000 Testing
- Optimizer: Adam (Learning rate $\alpha = 10^{-3}$)
- Loss Function: Mean Squared Error (MSE)
- Duration: 20 Epochs

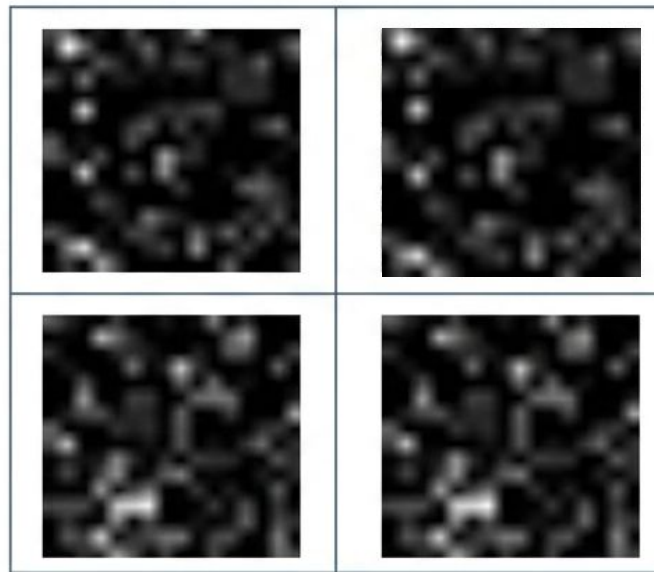


Security Analysis: Shadows as Pure Noise

Individual shares reveal absolutely no structural or visible information. Confidentiality is strictly maintained until all latent variables are recombined.



Original Image



Independent Shadows (S1-S4)

Individual Share Metrics: SSIM $\sim$ 0.05 to 0.10 | PSNR < 10 dB

Performance Analysis: High-Fidelity Reconstruction

Upon concatenation of all four shares, the decoder network successfully restores the spatial resolution with only negligible, pixel-level deviations.



Original

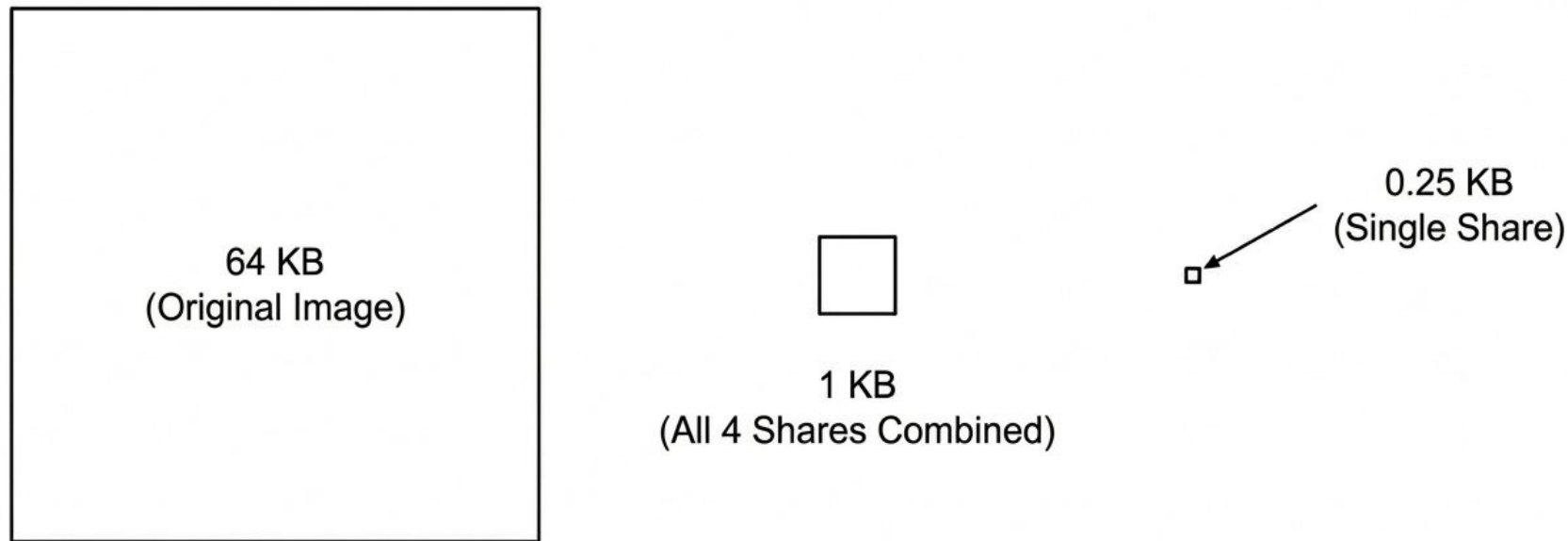


Reconstructed

Reconstruction Metrics: PSNR > 30 dB | SSIM ~ 1.0

98% Reduction

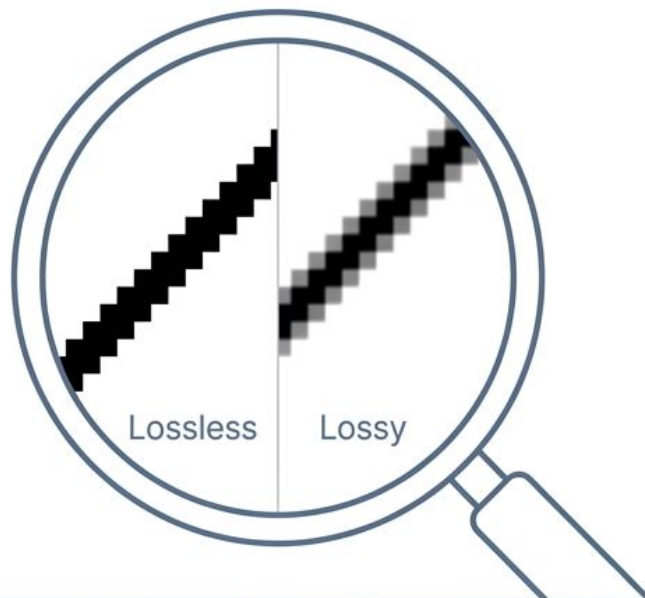
By compressing the image before splitting, the autoencoder bypasses the fundamental storage bottleneck of traditional cryptographic sharing.



98% Reduction in Storage Overhead

The Methodological Trade-off: Lossy vs. Lossless

Unlike traditional Boolean methods, the latent space compression relies on downsampling. This means the reconstruction is not perfectly lossless.



Latent space compression introduces minor smoothing artifacts. However, structural similarity remains > 0.99 , making the trade-off highly favorable for vast storage gains.

Publication Details

TITLE	VENUE	STATUS
Role-Aware Log Representations for Enhanced Anomaly Detection via Latent Token Role Induction	Lecture Notes in Networks and Systems, Springer Nature (CVR 2026)	Accepted
PoolLiteSIS: A Lightweight Secret Image Sharing Using Pooling-Based Dimensional Reduction	<i>Journal of Cyber Security Technology</i>	Submitted
Vulnerability Assessment of XOR-Based Secret Image Sharing Schemes Against Autoencoder Based Reconstruction Attacks	4th International Conference on Networks and Cryptology (NETCRYPT 2026)	Submitted
Cryptanalysis of a Medical Image Encryption Scheme Using Multiple Chaotic Maps	<i>Journal of Cryptologia</i>	Submitted
A Cryptanalysis of Multi-Chaos-Based Image Encryption for IoT Using Chosen Plaintext Attacks	<i>Journal of Cryptologia</i>	Submitted